SOFTWARE TESTING

CHEAT SHEETS

How to Test Software Well — Language-Agnostic, With pytest, Vitest, JUnit and Playwright Links

Test Pyramid · Test Types · Good Tests · AAA · Test Data · Test Smells · Test Doubles · Testability · TDD · BDD · Integration · Databases · API Tests · Contract Tests · E2E Strategy · Flaky Tests · Property-Based · Snapshots · Mutation · Fuzzing · Coverage · CI · Performance · Accessibility · Security · Legacy Code · Test Strategy · AI Agents

40 REFERENCE SHEETS

CONTENTS

FOUNDATIONS

01 Why We Test & What a Test Is
02 The Test Pyramid & Its Alternatives
03 Types of Tests
04 What Makes a Good Test

WRITING TESTS

05 Arrange-Act-Assert & Given-When-Then
06 Naming & Organizing Tests
07 Writing Useful Assertions
08 Test Data
09 Test Smells

TEST DOUBLES

10 Dummy, Stub, Fake, Spy & Mock
11 When (and When Not) to Mock
12 Designing for Testability

TDD & BDD

13 TDD: Red, Green, Refactor
14 TDD Styles
15 BDD & Gherkin

INTEGRATION & API TESTING

16 Integration Testing Basics
17 Databases in Tests
18 Testing HTTP APIs
19 Contract Testing

UI & END-TO-END TESTING

20 Component Tests vs End-to-End Tests
21 End-to-End Test Strategy
22 Flaky Tests

BEYOND EXAMPLE-BASED TESTS

23 Property-Based Testing
24 Snapshot & Approval Testing
25 Mutation Testing
26 Fuzz Testing

COVERAGE & SUITE HEALTH

27 Code Coverage
28 Test Suite Health

TESTING IN CI

29 Tests in the Pipeline
30 Parallelism & Sharding
31 Test Reports & Failure Triage

NON-FUNCTIONAL TESTING

32 Performance & Load Testing
33 Accessibility Testing
34 Security Testing Basics

TESTING IN PRACTICE

35 Testing Legacy Code
36 Testing Time, Async & Concurrency
37 A Test Strategy for a Project
38 Tests & AI Coding Agents

QUICK REFERENCE

39 Quick Reference — Test Types & Doubles
40 Quick Reference — Techniques & Smells

Pseudocode tests · In Your Stack links to the pytest, Jest & Vitest, JUnit and Playwright sets · Begin

TST

Table of Contents

40 sheets · syntax · examples · notes

FOUNDATIONS 01 – 04

01 Why We Test & What a…
02 The Test Pyramid & Its …
03 Types of Tests
04 What Makes a Good Te…

WRITING TESTS 05 – 09

05 Arrange-Act-Assert & …
06 Naming & Organizing T…
07 Writing Useful Assertio…
08 Test Data
09 Test Smells

TEST DOUBLES 10 – 12

10 Dummy, Stub, Fake, Sp…
11 When (and When Not) …
12 Designing for Testability

TDD & BDD 13 – 15

13 TDD: Red, Green, Refac…
14 TDD Styles
15 BDD & Gherkin

INTEGRATION & API TESTING 16 – 19

16 Integration Testing Bas…
17 Databases in Tests
18 Testing HTTP APIs
19 Contract Testing

UI & END-TO-END TESTING 20 – 22

20 Component Tests vs E…
21 End-to-End Test Strat…
22 Flaky Tests

BEYOND EXAMPLE-BASED TESTS 23 – 26

23 Property-Based Testing
24 Snapshot & Approval T…
25 Mutation Testing
26 Fuzz Testing

COVERAGE & SUITE HEALTH 27 – 28

27 Code Coverage
28 Test Suite Health

TESTING IN CI 29 – 31

29 Tests in the Pipeline
30 Parallelism & Sharding
31 Test Reports & Failure …

NON-FUNCTIONAL TESTING 32 – 34

32 Performance & Load T…
33 Accessibility Testing
34 Security Testing Basics

TESTING IN PRACTICE 35 – 38

35 Testing Legacy Code
36 Testing Time, Async & …
37 A Test Strategy for a Pr…
38 Tests & AI Coding Age…

QUICK REFERENCE 39 – 40

39 Quick Reference — Tes…
40 Quick Reference — Tec…

Sheet 01

Why We Test & What a Test Is

confidence to change code · regression safety · tests as documentation · cost of a bug by stage · anatomy of a test

TESTS BUY THE FREEDOM TO CHANGE CODE

An **automated test** runs a piece of code with known inputs and checks the result, every time, in seconds. The real payoff isn't finding today's bug; it's **confidence**: you can refactor, upgrade a library or add a feature and know within minutes whether something that used to work broke (a **regression**). Good tests also **document** behavior: they show how code is meant to be used, and they stay correct because they run.

NOTES

- A bug found while coding costs minutes; the same bug in production can cost days
- Tests don't prove code correct; they reduce risk where you point them
- Manual testing still matters for exploring; automate what you check repeatedly
- This set uses pseudocode; In Your Stack links to real syntax

ANATOMY OF A TEST (HOUSE STYLE, SHEET 05)

```
test "cart total applies a 25% discount":
    # Arrange — build the situation
    cart = Cart()
    cart.add(item="apple", price=10.00, qty=2)

    # Act — do the one thing under test
    total = cart.total(discount=0.25)

    # Assert — check the outcome
    assert total == 15.00

# A test runner finds tests, runs each in isolation,
# and reports: 1 passed, 0 failed (0.01 s)
```

CODE EXPLAINED

→ The name states the behavior, so a failure reads as a broken promise

→ Arrange-Act-Assert keeps setup, action and check visibly separate

→ One clear outcome per test makes failures easy to diagnose

→ Runs in milliseconds, so it can run on every save and every push

IN YOUR STACK

- **pytest** `def test_…():` + plain `assert` ↗01
- **Vitest** `it("…", () => expect(x).toBe(y))` ↗01
- **JUnit** `@Test void …()` + `assertEquals` ↗01
- **Playwright** `test("…", async ({ page }) => …)` for the browser ↗01

SMELL CHECK

- **You see** a suite nobody runs before merging → **it means** tests aren't trusted or are too slow → **do** make them fast and required in CI
- **You see** tests added only after bugs → **it means** no safety net while building → **do** test as you write the feature

WHEN A BUG IS FOUND

STAGE	TYPICAL COST
While writing code	Minutes
In code review / CI	An hour
In QA / staging	Hours, a context switch
In production	Days, plus user impact and trust
Never (silent data bug)	Unbounded

WHAT TESTS GIVE YOU

BENEFIT	BECAUSE
Regression safety	Old behavior is rechecked on every change
Refactoring	Change structure with proof behavior held
Documentation	Examples that can't go stale
Design feedback	Hard-to-test code reveals coupling
Faster debugging	A failing test points at the change

⚠ COMMON MISTAKES

- Treating a passing suite as proof the software is correct
- Writing tests only to hit a coverage number
- Skipping tests "to go faster" and paying for it in debugging time later

✓ TIPS & BEST PRACTICES

- Start with the code that would hurt most if it broke
- Keep tests fast enough to run constantly
- Read a test as documentation: could a newcomer learn the behavior from it?

QUICK SUMMARY

- tests — confidence to change code
- regressions — caught in minutes, not production
- tests as docs — always up to date
- Arrange-Act-Assert — the basic shape

The Test Pyramid & Its Alternatives

unit / integration / E2E layers · speed vs confidence · testing trophy · honeycomb · ice-cream cone anti-pattern

HOW MANY TESTS AT EACH LEVEL

The **test pyramid** says: many fast **unit** tests at the bottom, fewer **integration** tests in the middle, a handful of slow **end-to-end** tests at the top. Lower tests are cheap and precise; higher tests give more confidence that the pieces work together but are slower and flakier. The **testing trophy** (front-end) and **honeycomb** (microservices) shift weight toward integration tests. The **ice-cream cone** is the anti-pattern: mostly manual and E2E.

NOTES

- The shape is a guideline about cost and feedback speed, not a quota
- Static checks (types, linters) form the trophy's base
- Pick the lowest level that can catch the bug
- Each layer should test what the layers below can't

INTERVIEW ANGLE

- ? **Q:** "Describe the test pyramid." Explain the three layers and the cost / speed / confidence trade-off
- ? Say *why* it's shaped that way: cheap, fast feedback at the bottom
- ? Mention when the trophy fits better (UI apps, thin logic) — shows judgment, not dogma

ONE FEATURE, THREE LEVELS

```
# Feature: "checkout applies a coupon"

# UNIT — pure logic, milliseconds, many of these
test "coupon SAVE10 takes 10% off":
    assert apply_coupon(total=50.00, code="SAVE10") == 45.00

# INTEGRATION — real pieces wired together, a real DB
test "checkout endpoint stores discounted order":
    response = http_post("/checkout", { cart_id: 7, coupon: "SAVE10" })
    assert response.status == 201
    assert db.orders.find(7).total == 45.00

# END-TO-END — real browser, whole app, few of these
test "shopper can use a coupon at checkout":
    open_page("/cart")
    fill("Coupon", "SAVE10"); click("Apply")
    expect(text("Total: $45.00")).visible
```

CODE EXPLAINED

- → Edge cases (expired, unknown, stacked coupons) belong in fast unit tests
- → The integration test proves the endpoint, logic and database agree
- → One E2E test proves a user can actually do it; it doesn't re-test every edge case
- → If the E2E test fails but unit tests pass, the bug is in the wiring

IN YOUR STACK

- **pytest** unit + integration (`TestClient` , `django_db`) ↗21
- **Vitest** unit, component (Testing Library), browser mode ↗19
- **JUnit** unit, slices (`@WebMvcTest`), `@SpringBootTest` ↗20
- **Playwright** the E2E layer (and API tests) ↗01

SMELL CHECK

- **You see** 300 E2E tests and 20 unit tests → **it means** an ice-cream cone: slow, flaky feedback → **do** push edge cases down to unit tests
- **You see** every unit test mocking everything → **it means** no proof the pieces fit → **do** add integration tests at real boundaries

LAYERS COMPARED

LAYER	SPEED	SCOPE	TYPICAL COUNT
Static (types, lint)	Instant	Whole code base	Always on
Unit	ms	One function / class	Hundreds–thousands
Integration	10 ms–1 s	Several parts + real infra	Dozens–hundreds
End-to-end	Seconds	Whole system via UI / API	Handful–dozens

SHAPES

SHAPE	EMPHASIS	FITS
Pyramid	Unit	Back ends, libraries
Trophy	Static + integration	UI apps (Kent C. Dodds)
Honeycomb	Integration	Microservices (Spotify)
Ice-cream cone	Manual + E2E	Anti-pattern

⚠ COMMON MISTAKES

- Testing every edge case through the UI
- Counting tests per layer instead of asking what each test catches
- Calling a test "unit" while it hits a real network service

✓ TIPS & BEST PRACTICES

- Ask "what's the cheapest test that would catch this bug?"
- Keep E2E tests to critical user journeys
- Add integration tests at the seams where bugs actually happen (DB, HTTP, queues)

QUICK SUMMARY

- unit — many, fast, precise
- integration — real pieces together
- E2E — few, slow, high confidence
- trophy / honeycomb — more integration

Types of Tests

unit · integration · end-to-end · contract · smoke · regression · acceptance · exploratory

NAMES FOR WHAT A TEST COVERS AND WHY

Test types describe **scope** (how much of the system a test touches) or **purpose** (why it exists). Scope: **unit**, **integration**, **end-to-end**. Purpose: **smoke** (does it start at all?), **regression** (did a fixed bug come back?), **acceptance** (does it meet the requirement?), **contract** (do services agree on an API?). **Exploratory** testing is a person probing the app without a script. Teams use the words differently, so agree on definitions.

NOTES

- "Unit" can mean one function or one module; define it for your team
- A regression test is usually added with the bug fix it guards
- Smoke tests run right after deploy and must be fast
- Tags / marks let one suite run different types at different times

INTERVIEW ANGLE

- ? **Q:** "Unit vs integration vs end-to-end?" Define each by what it touches and what it can catch
- ? Say where *you* draw the unit / integration line (e.g. "anything that touches the database is integration")
- ? Mention contract tests for services — it signals experience

SAME PRODUCT, DIFFERENT PURPOSES

```
# SMOKE — after deploy: is the app alive?
test "health endpoint is up" [tag: smoke]:
    assert http_get("/health").status == 200

# REGRESSION — bug #142 must never return
test "order with 0.1 + 0.2 discount rounds to cents" [tag: regression]:
    assert order_total([10.00], discount=0.3) == 7.00

# ACCEPTANCE — written from the requirement
test "guest can check out without an account" [tag: acceptance]:
    given a guest with an item in the cart
    when they check out with an email only
    then an order confirmation is emailed

# CONTRACT — consumer's expectation of a provider API
contract "orders-ui expects GET /orders/7":
    response has { id: number, status: "NEW" | "PAID" }
```

CODE EXPLAINED

- → Tags let you run smoke tests on deploy and the full suite on every PR
- → Regression tests name the bug they guard so nobody deletes them casually
- → Acceptance tests often use Given/When/Then (sheet 15)
- → Contract tests check API shape without running both services together (sheet 19)

IN YOUR STACK

- **pytest** `@pytest.mark.smoke` + `-m smoke` ↗12
- **Vitest** projects or name filters per type ↗21
- **JUnit** `@Tag("smoke")` + build tag filters ↗06
- **Playwright** `{ tag: "@smoke" }` + `--grep @smoke` ↗07

SMELL CHECK

- **You see** "unit tests" that need Docker → **it means** mislabeled integration tests → **do** tag them and run them separately
- **You see** a bug fixed with no test → **it means** it can quietly return → **do** add a regression test in the same PR

BY SCOPE

TYPE	TOUCHES
Unit	One function / class, no I/O
Integration	Several parts + real DB / HTTP / queue
End-to-end	Whole deployed system via UI or API
Component	One UI component rendered in isolation

BY PURPOSE

TYPE	ANSWERS	WHEN IT RUNS
Smoke	Is it alive?	After every deploy
Regression	Did an old bug return?	Every run
Acceptance	Does it meet the requirement?	PR / release
Contract	Do services agree on the API?	Both sides' CI
Performance	Is it fast enough? (sheet 32)	Nightly / pre-release
Exploratory	What did we not think of?	By people, any time

⚠ COMMON MISTAKES

- Arguing about labels instead of what each test catches
- Running slow integration tests in the "fast" job because they're called unit tests
- Dropping manual exploratory testing entirely

✓ TIPS & BEST PRACTICES

- Write down your team's definitions in the repo's testing guide
- Tag tests by type so CI can run them on different schedules
- Name regression tests after the issue number

QUICK SUMMARY

- scope — unit · integration · E2E
- purpose — smoke · regression · acceptance · contract
- exploratory — humans probing
- tags — run types at different times

What Makes a Good Test

FIRST (fast, isolated, repeatable, self-validating, timely) · deterministic · readable · behavior over implementation · one reason to fail

FAST, TRUSTWORTHY, READABLE, BEHAVIOR-FOCUSED

A good test is **fast**, **isolated** from other tests, **repeatable** (same result every run), **self-validating** (pass or fail, no reading logs), and written **timely**, close to the code: the **FIRST** acronym. It tests **behavior** (what the code promises) rather than implementation details (how it does it today), so refactoring doesn't break it. And it fails for **one reason** you can read from its name.

NOTES

- A test that sometimes fails is worse than no test: people learn to ignore red
- Test the public API of a unit, not its private helpers
- Readable beats clever: a test is read far more often than written
- "Would this test survive a refactor that keeps behavior?"

INTERVIEW ANGLE

- ? **Q:** "What makes a good unit test?" Walk through FIRST
- ? Add *behavior, not implementation* with an example (testing output vs asserting private calls)
- ? Mention determinism: no real clock, network or randomness

IMPLEMENTATION-COUPLED VS BEHAVIOR-FOCUSED

```
# ✗ coupled to implementation: breaks on any refactor
test "total calls _sum_lines then _apply_tax":
    cart = Cart()
    spy(cart, "_sum_lines"); spy(cart, "_apply_tax")
    cart.total()
    assert cart._sum_lines.called_once
    assert cart._apply_tax.called_after(cart._sum_lines)

# ✓ tests the promise: survives refactors
test "total includes 7% tax":
    cart = Cart(tax_rate=0.07)
    cart.add(price=100.00)
    assert cart.total() == 107.00

# ✗ not repeatable: depends on today's date
test "trial is active":   assert Trial(start=today()).active()
# ✓ repeatable: fixed clock (sheet 12)
test "trial is active on day 13":
    clock = FixedClock("2026-11-13")
    assert Trial(start="2026-11-01", clock=clock).active()
```

CODE EXPLAINED

- → The first test fails if you merge the two helpers, even though nothing a user sees changed
- → The second test pins down the observable result
- → Real dates make tests pass today and fail next month
- → Injecting a clock makes time a test input like any other

IN YOUR STACK

- **pytest** plain functions + fixtures; `monkeypatch` the clock ↗13
- **Vitest** `vi.setSystemTime` for time ↗14
- **JUnit** inject `Clock`; AssertJ for readable checks ↗09
- **Playwright** web-first assertions avoid timing flakes ↗18

SMELL CHECK

- **You see** tests that break on every refactor → **it means** they test implementation → **do** assert on outputs and public behavior
- **You see** "rerun and it passes" → **it means** non-determinism → **do** remove real time, randomness and shared state

FIRST

LETTER	MEANS	BREAKS WHEN
F — Fast	ms per unit test	Real network / sleep calls
I — Isolated	No order dependence	Shared mutable state
R — Repeatable	Same result anywhere	Clock, randomness, env
S — Self-validating	Pass / fail by itself	"Check the log output"
T — Timely	Written with the code	Tests added months later

BEHAVIOR VS IMPLEMENTATION

TEST THIS	NOT THIS
Return values	Private methods
State changes others can see	Internal call order
Calls across boundaries (email sent)	Calls between your own classes
Errors raised for bad input	Which helper raised them

⚠ COMMON MISTAKES

- Making private methods public just to test them
- Asserting on log messages instead of results
- Tests that pass alone but fail in the full suite

✓ TIPS & BEST PRACTICES

- If a test is hard to write, the design may need a seam (sheet 12)
- Run tests in random order to catch hidden dependencies
- Name tests so a failure reads like a broken requirement

QUICK SUMMARY

- FIRST — fast, isolated, repeatable, self-validating, timely
- deterministic — no real clock / network / randomness
- behavior over implementation
- one reason to fail

Arrange-Act-Assert & Given-When-Then

AAA structure · Given-When-Then · one act per test · setup vs test body · house style for this set

A SHAPE EVERY READER RECOGNIZES

Arrange-Act-Assert splits a test into three parts: build the situation, perform **one** action, check the outcome. **Given-When-Then** is the same idea in behavior language. Keeping the parts visibly separate (blank lines or comments) makes any test readable at a glance, and a single Act keeps the reason for a failure obvious. This sheet also defines the pseudocode style used throughout the set.

NOTES

- If you need two Acts, you probably have two tests
- Shared Arrange code moves to fixtures / setup, not copy-paste
- Asserting several facts about **one** outcome is fine
- Avoid conditionals and loops in tests; they hide which path ran

HOUSE STYLE FOR THIS SET

```
test "<behavior in plain words>":        # one test, one behavior
    # Arrange
    user   = make_user(role="admin")       # factories (sheet 08)
    clock = FixedClock("2026-11-04")       # injected dependencies
    mailer = fake Mailer()                 # doubles named by role (sheet 10)

    # Act
    result = invite(user, "bo@example.com", clock, mailer)

    # Assert
    assert result.status == "SENT"
    assert mailer.sent_to == ["bo@example.com"]

expect(x).visible       # UI / async assertion that waits
raises Error: f(x)      # assert that f(x) raises
given / when / then     # behavior-style steps (sheet 15)
```

CODE EXPLAINED

- → `make_…` builds test data with defaults (sheet 08)
- → `fake`, `stub`, `mock` say what kind of double an object is
- → Two asserts about the same result are one logical check
- → The real syntax for each line is one click away in In Your Stack

IN YOUR STACK

- **pytest** blank lines between parts; `pytest.raises` ↗22
- **Vitest** `it()` blocks; `expect(...).rejects` ↗04
- **JUnit** comments or `// given // when // then` ↗04
- **Playwright** `test.step("…")` labels each phase in reports ↗08

SMELL CHECK

- **You see** Act → Assert → Act → Assert in one test → **it means** several tests glued together → **do** split them
- **You see** 40 lines of Arrange → **it means** missing factories or a design smell → **do** extract builders or simplify the API

AAA VS GIVEN-WHEN-THEN

AAA	GWT	CONTAINS
Arrange	Given	Objects, data, doubles, fixed time
Act	When	Exactly one action
Assert	Then	Checks on the outcome

RULES OF THUMB

RULE	WHY
One Act per test	One reason to fail
No logic (if / for)	Every path is explicit
Literal expected values	Readers can check them
Shared setup in fixtures	Tests stay short
Blank line between parts	Structure visible at a glance

⚠ COMMON MISTAKES

- Hiding the Act inside a helper so readers can't see what's tested
- Assertions in setup code that fail before the test starts
- Using `if` in a test to choose expectations

✓ TIPS & BEST PRACTICES

- Read only the Act line: does it match the test name?
- Put the expected value as a literal in the Assert
- Use parameterized tests instead of loops

QUICK SUMMARY

- Arrange — situation
- Act — one action
- Assert — outcome
- Given / When / Then — same idea, behavior words

Naming & Organizing Tests

naming patterns (`should…when…`) · one behavior per test · grouping by unit / feature · file layout · test-to-code mapping

FIND THE TEST, READ THE FAILURE

A test name should state the **behavior** and the **situation**, so a failure report reads like a broken requirement: "rejects an expired coupon". Group tests the way readers look for them: by **unit** (one file per module) or by **feature** (one file per user-facing capability). Mirror the source tree so every code file has an obvious test file.

NOTES

- Names are for humans; long and specific beats short and vague
- Nested groups (describe / @Nested / classes) express situations
- Keep fixtures next to the tests that use them
- Pick one naming pattern per repo and stick to it

NAMES THAT READ AS REQUIREMENTS

```
# ✗ vague
test "test1"
test "coupon"
test "testApplyCouponWorks"

# ✓ behavior + situation
group "apply_coupon":
    group "when the coupon is valid":
        test "takes 10% off the subtotal"
        test "records the coupon on the order"
    group "when the coupon is expired":
        test "rejects it with COUPON_EXPIRED"
        test "leaves the total unchanged"

# file layout mirrors the code
src/checkout/coupons.py      →  tests/checkout/test_coupons.py
src/cart/Cart.java           →  src/test/java/cart/CartTest.java
src/cart/cart.ts             →  src/cart/cart.test.ts
```

CODE EXPLAINED

→ A failure reads "apply_coupon › when the coupon is expired › rejects it with COUPON_EXPIRED"

→ Groups hold shared setup for their situation

→ "Works" says nothing about what working means

→ Mirrored paths make the test for any file easy to find

IN YOUR STACK

- **pytest** `test_*.py` mirroring `src/`; classes as groups ↗03
- **Vitest** `describe` nesting; `*.test.ts` next to the code ↗04
- **JUnit** `@Nested` + `@DisplayName` ↗06
- **Playwright** `test.describe` per page or flow ↗06

SMELL CHECK

- **You see** a test called `testProcess` with 12 asserts → **it means** an unnamed bundle of behaviors → **do** split and name each one
- **You see** helpers duplicated across test files → **it means** no shared fixtures → **do** move them to a fixtures / conftest / support module

NAMING PATTERNS

PATTERN	EXAMPLE
`Behavior`	rejects an expired coupon
`should…when…`	should reject coupon when expired
`given…when…then…`	given expired coupon, when applied, then rejected
`method_state_result`	applyCoupon_expired_throws

ORGANIZING OPTIONS

BY	GOOD FOR	WATCH OUT
`Unit / module`	Libraries, back-end code	Feature tests spread out
`Feature`	Apps, BDD	Unit tests harder to find
`Test type folder`	Separating slow tests	Duplicate trees

⚠ COMMON MISTAKES

- Naming tests after the method instead of the behavior
- One giant test file per package
- Test files that drift from the code layout after a refactor

✓ TIPS & BEST PRACTICES

- Read the test report like a spec; rename tests that don't make sense
- Use groups for situations, test names for outcomes
- Move or rename test files in the same commit as the code they cover

QUICK SUMMARY

- name = behavior + situation
- groups = situations
- mirror the source tree
- one pattern per repo

Writing Useful Assertions

ASSERTIONS DECIDE WHAT A TEST PROVES

An assertion should be **exact enough** to catch the bug you care about and **loose enough** to ignore details that don't matter (generated IDs, timestamps, order of a set). Compare whole values for a single readable diff, use tolerance for floats, and name what you check. Many unlabeled asserts in one test, where you can't tell which failed, is called **assertion roulette**.

NOTES

- `assert result` (truthy) proves very little
- Order matters for lists, not for sets: use the right comparison
- Floats need a tolerance; money is better as integer cents or decimals
- Custom matchers give domain-specific failure messages

PRECISE, READABLE CHECKS

```
# ✗ too loose — passes for almost anything
assert response
assert len(orders) > 0

# ✓ exact on what matters, flexible on the rest
assert response.status == 201
assert response.body == {
    id:        any(String),          # generated — don't pin it
    createdAt: any(Date),
    status:    "NEW",
    total:     45.00,
}

assert approx(order.tax, 3.15, abs=0.005)       # floats
assert sorted(tags) == ["gift", "rush"]          # order doesn't matter
assert lines == [("A1", 2), ("B2", 1)]           # order matters

# ✗ roulette: which one failed?
assert a; assert b; assert c; assert d
# ✓ grouped with labels, all reported
assert_all("address", street == "1 Main", city == "Tripoli")
```

CODE EXPLAINED

→ Comparing the whole body gives one diff showing every wrong field

→ `any(String)` keeps the test stable without ignoring the field entirely

→ Sort or use a set comparison when order isn't part of the contract

→ Grouped assertions report every failure, not just the first

IN YOUR STACK

- **pytest** plain `assert` diffs; `pytest.approx` ↗04
- **Vitest** `toEqual` + `expect.any`; `expect.soft` ↗09
- **JUnit** AssertJ `usingRecursiveComparison`, `assertAll` ↗09
- **Playwright** web-first `toHaveText` / `toBeVisible` retry ↗18

SMELL CHECK

- **You see** `assertTrue(x.equals(y))` → **it means** the failure will say only "expected true" → **do** use an equality assertion that prints both values
- **You see** tests asserting on a timestamp → **it means** flaky by design → **do** freeze time or use a matcher

ASSERTION KINDS

NEED	USE
Exact value	equality
Shape with generated fields	partial / asymmetric matchers
Float	approx / tolerance
Unordered collection	set / any-order comparison
Exception	raises / assertThrows
Eventually true	polling / web-first assertion

WEAK ASSERTIONS

WEAK	STRONGER
`assert result`	assert result == expected
`len(x) > 0`	x == [expected items]
`assertTrue(a.equals(b))`	assertEquals(b, a)
`no exception thrown`	assert on the returned value
`ignore field X`	matcher for X (any type)

⚠ COMMON MISTAKES

- Asserting only that no exception was thrown
- Copying the implementation into the test to compute the expected value
- Ignoring whole fields to make a flaky test pass

✓ TIPS & BEST PRACTICES

- Write the expected value as a literal the reader can check by eye
- Mutation testing (sheet 25) shows which assertions are too weak
- Add a custom matcher when the same complex check appears in many tests

QUICK SUMMARY

- exact on what matters, loose on the rest
- whole-value equality for one diff
- approx for floats
- avoid assertion roulette

Test Data

MAKE THE IMPORTANT DATA OBVIOUS

Test data should show **only what matters** to the test and default everything else. A **factory** or **builder** creates a valid object with sensible defaults, and the test overrides just the fields it cares about. An **object mother** offers named, ready-made objects (`premiumCustomer()`). Random data can find surprises, but always log the **seed**. **Golden files** hold large expected outputs.

NOTES

- Mystery data ("why is age 37?") makes tests hard to read
- Factories keep tests valid when a required field is added
- Avoid sharing one mutable fixture across tests
- Keep fixture files small and named for the scenario

FACTORIES AND BUILDERS

```
# a factory with defaults: tests override only what matters
make_order(**overrides):
    defaults = { customer: make_customer(), status: "NEW",
                 lines: [line(sku="A1", qty=1, price=10.00)],
                 created_at: "2026-11-04T09:00:00Z" }
    return Order(defaults | overrides)

test "paid orders can't be cancelled":
    order = make_order(status="PAID")          # the one fact that matters
    raises CannotCancel: cancel(order)

# builder for complex objects
order = OrderBuilder().for_customer("ann").with_line("A1", 2).paid().build()

# object mother for well-known personas
customer = Customers.premium_in_germany()

# random data: reproducible with a seed
rng = Random(seed=env("TEST_SEED", default=1234))
email = fake.email(rng)
```

CODE EXPLAINED

→ The test reads "a PAID order"; everything else is a valid default

→ Adding a required field later only changes the factory, not every test

→ Builders read well for objects with many optional parts

→ Logging and reusing the seed makes a random failure reproducible

IN YOUR STACK

- **pytest** factory fixtures; factory_boy ↗08
- **Vitest** `test.extend` fixtures; builder functions ↗05
- **JUnit** `@MethodSource` + builders; Instancio ↗12
- **Playwright** custom fixtures per role / data ↗24

SMELL CHECK

- **You see** a 30-field object literal in every test → **it means** no factory → **do** add one with defaults
- **You see** tests reading one shared `test_data.json` → **it means** hidden coupling between tests → **do** give each test its own data

DATA PATTERNS

PATTERN	BEST FOR
`Literal values`	Small, obvious inputs
`Factory with defaults`	Most domain objects
`Builder`	Objects with many optional parts
`Object mother`	Named personas reused widely
`Fixture / seed file`	Reference data (countries, plans)
`Golden file`	Large expected outputs

RANDOM DATA RULES

RULE	WHY
`Log the seed`	Reproduce failures
`Allow fixing the seed`	Debug locally
`Keep values valid`	Test the logic, not the generator
`Prefer property-based tools`	Shrinking finds minimal cases

⚠ COMMON MISTAKES

- Unexplained magic numbers in test data
- Production data dumps in test fixtures (privacy and size)
- Random data without a logged seed

✓ TIPS & BEST PRACTICES

- Name test data after the scenario: `expired_coupon`, not `coupon2`
- Keep factories next to the domain model they build
- Use property-based testing (sheet 23) when you want lots of varied data on purpose

QUICK SUMMARY

- factories — valid defaults, override what matters
- builders / object mothers — readable complex data
- seeds — reproducible randomness
- golden files — large expected output

Test Smells

SIGNS A TEST SUITE IS GETTING SICK

A **test smell** is a pattern that makes tests hard to trust, read or maintain. **Fragile** tests break on harmless refactors; **slow** tests stop people running them; a **mystery guest** depends on data you can't see in the test; **logic in tests** (ifs, loops) hides what ran; an **eager test** checks too many behaviors; **over-specification** asserts details nobody cares about; **shared mutable state** makes order matter.

NOTES

- Smells are hints, not rules; judge the cost
- Every Smell Check box in this set is a smell tied to that topic
- Lint rules catch some smells automatically
- Fix smells when you touch a test; don't schedule a rewrite

INTERVIEW ANGLE

- ? **Q:** "How do you spot a bad test?" Name two or three smells with a fix each
- ? Fragile (coupled to implementation) and flaky (non-deterministic) are the two interviewers expect
- ? Mention that a test which can't fail is the worst smell of all

SMELLS AND FIXES

```
# MYSTERY GUEST — where does user 42 come from?
test "admin sees reports":   assert can_view(load_user(42), "reports")
# fix: build it here
test "admin sees reports":   assert can_view(make_user(role="admin"), "reports")

# LOGIC IN TEST — which branch ran?
test "discounts":
    for code in codes:
        if code.startswith("VIP"): assert apply(code) < 0.8
        else:                      assert apply(code) == 0.9
# fix: parameterized cases with explicit expectations (sheet 05)

# OVER-SPECIFICATION — asserts every call
verify(repo).open(); verify(repo).begin(); verify(repo).save(o); verify(repo).commit()
# fix: assert the outcome
assert repo.find(o.id) == o

# THE TEST THAT CAN'T FAIL
test "sends email":
    try: send_welcome(user)
    except: pass                        # swallows every failure
```

CODE EXPLAINED

- → Building the admin in the test makes the dependency visible
- → Loops and ifs mean nobody knows which case broke
- → Verifying every internal call breaks whenever the implementation changes
- → A swallowed exception makes the test pass no matter what

IN YOUR STACK

- **pytest** Ruff `PT` rules (flake8-pytest-style) ↗22
- **Vitest** `@vitest/eslint-plugin` rules ↗04
- **JUnit** Sonar / PMD test rules ↗04
- **Playwright** `eslint-plugin-playwright` (no waits, no conditionals) ↗06

SMELL CHECK

- ⚑ **You see** a test that passed when you deleted the code under test → **it means** it can't fail → **do** break the code on purpose and watch it go red
- ⚑ **You see** tests that only fail in the full run → **it means** shared mutable state → **do** isolate setup per test

SMELL CATALOG

SMELL	SYMPTOM	FIX
Fragile	Breaks on refactor	Test behavior
Flaky	Random failures	Remove nondeterminism (sheet 22)
Slow	Nobody runs it	Push down the pyramid
Mystery guest	Hidden data / files	Build data in the test
Eager test	Many behaviors in one	Split it
Logic in test	if / for inside	Parameterize
Over-specified	Asserts incidental details	Assert outcomes
Shared state	Order-dependent	Fresh setup per test

FINDING SMELLS

TECHNIQUE	FINDS
Code review of tests	Mystery guests, logic, eager tests
Lint rules	Common anti-patterns
Random order runs	Shared state
Slowest-tests report	Slow tests
Mutation testing	Weak or missing assertions

⚠ COMMON MISTAKES

- Rewriting the whole suite instead of fixing smells as you go
- Adding retries to hide flakiness
- Ignoring smells in tests because "it's only test code"

✓ TIPS & BEST PRACTICES

- Review test code as carefully as production code
- Break the code on purpose before trusting a new test
- Track slow and flaky tests in CI reports

QUICK SUMMARY

- fragile / flaky / slow — trust killers
- mystery guest — hidden data
- logic in tests — hidden paths
- can't-fail tests — the worst smell

Sheet 10

Dummy, Stub, Fake, Spy & Mock

the five doubles · state vs interaction verification · which to reach for · naming doubles by role

STAND-INS FOR REAL COLLABORATORS

A **test double** replaces a real dependency in a test. A **dummy** is passed but never used. A **stub** returns canned answers. A **fake** is a working lightweight version (an in-memory repository). A **spy** records how it was called. A **mock** is set up with expectations and checks them. Libraries call everything a "mock", but knowing which you need keeps tests simpler.

NOTES

- Stubs and fakes support **state** verification (check results)
- Spies and mocks support **interaction** verification (check calls)
- Fakes are often worth sharing across the suite
- Terms from Gerard Meszaros, *xUnit Test Patterns*

INTERVIEW ANGLE

- ? **Q:** "Mock vs stub vs fake?" Give a one-line definition each
- ? Stub = canned answer, fake = working shortcut, mock = expectations on calls
- ? Example: stub a price API, fake a repository in memory, mock the email sender to verify it was called

ONE OF EACH

```
# DUMMY — required parameter, never used
report = make_report(data, logger=DummyLogger())

# STUB — canned answer
rates = stub RateService { get("EUR") -> 1.1 }
assert convert(10, "EUR", rates) == 11.0

# FAKE — real behavior, simpler implementation
repo = InMemoryOrderRepository()
service = OrderService(repo)
service.place(cart)
assert repo.count() == 1

# SPY — records calls, then you check
mailer = spy Mailer()
service.place(cart, mailer)
assert mailer.calls == [("send", "ann@x.com", "Order received")]

# MOCK — expectations set up front
mailer = mock Mailer() expecting send("ann@x.com", any) once
service.place(cart, mailer)
mailer.verify()
```

CODE EXPLAINED

- → The stub only answers questions; nobody checks how it was used
- → The fake behaves like a real repository, so tests check final state
- → Spies check after the fact; mocks declare expectations first (most libraries blur this)
- → Prefer state checks; use interaction checks for side effects like email

IN YOUR STACK

- **pytest** stub with `monkeypatch`; `Mock` / `mocker.spy` ↗14
- **Vitest** `vi.fn()` stubs and spies ↗11
- **JUnit** Mockito `when` (stub), `verify` (mock), `@Spy` ↗17
- **Playwright** `page.route` stubs network responses ↗29

SMELL CHECK

- **You see** mocks for value objects and DTOs → **it means** doubling things that are cheap to build → **do** use real instances
- **You see** a mock set up with 15 `when` calls → **it means** a fat interface or the wrong boundary → **do** use a fake or narrow the dependency

THE FIVE DOUBLES

DOUBLE	DOES	VERIFY BY
Dummy	Fills a parameter	—
Stub	Returns canned data	State
Fake	Works, but simplified	State
Spy	Records calls (may call real code)	Interaction, after
Mock	Pre-programmed expectations	Interaction

WHICH ONE?

NEED	REACH FOR
Fill an unused parameter	Dummy
Control what a dependency returns	Stub
Realistic behavior without I/O	Fake
Check a side effect happened	Spy / mock
Cheap, deterministic collaborator	The real object

⚠ COMMON MISTAKES

- Mocking everything by reflex
- Verifying stubbed calls (`when x` then `verify x`) — redundant
- Fakes that drift from the real behavior without their own tests

✓ TIPS & BEST PRACTICES

- Test your fakes against the same contract tests as the real thing
- Name doubles by role: `fake_repo`, `stub_rates`, `spy_mailer`
- Prefer state verification when you can

QUICK SUMMARY

- dummy · stub · fake · spy · mock
- state verification — stubs, fakes
- interaction verification — spies, mocks
- real objects when they're cheap

When (and When Not) to Mock

mock at boundaries · don't mock what you don't own · ports & adapters · over-mocking · mocks that test the mock

MOCK THE EDGES, NOT THE MIDDLE

Doubles belong at **boundaries**: the network, the clock, the file system, payment gateways, email. Inside your own code, use real objects. **Don't mock what you don't own**: instead of mocking a third-party SDK's many methods, wrap it in a small **port** (interface) you own, mock that, and test the **adapter** against the real thing in an integration test. Over-mocking produces tests that pass while the real system is broken.

NOTES

- A mock encodes your assumption about a dependency; if it's wrong, the test lies
- Integration tests check those assumptions
- Network-level fakes (MSW, WireMock, `page.route`) avoid mocking client libraries
- Classicist vs mockist styles: sheet 14

INTERVIEW ANGLE

- ? **Q:** "When shouldn't you mock?" — your own domain objects, value objects, and libraries you don't own
- ? Explain ports and adapters: mock your interface, integration-test the adapter
- ? Mention that heavy mocking makes refactoring painful

WRAP, THEN MOCK YOUR OWN PORT

```
# ✗ mocking a third-party SDK directly
stripe = mock StripeSDK
stripe.customers.create(...) returns { id: "cus_1" }
stripe.payment_intents.create(...) returns { status: "succeeded" }
# … 6 more stubs mirroring Stripe's API, wrong in subtle ways

# ✓ a small port you own
interface PaymentGateway:
    charge(customer_email, amount_cents) -> Receipt

# unit tests: fake the port
gateway = FakePaymentGateway(always="succeeded")
checkout = Checkout(gateway)
assert checkout.pay(cart).status == "PAID"

# integration test: the real adapter against Stripe's test mode
test "StripeGateway charges in test mode" [tag: integration]:
    receipt = StripeGateway(test_key).charge("ann@x.com", 1999)
    assert receipt.status == "succeeded"
```

CODE EXPLAINED

- → Mocking the SDK copies its API into every test and drifts from reality
- → The port describes what *your* app needs, in your terms
- → Unit tests use a fake gateway and stay fast
- → One integration test proves the adapter works with the real service

IN YOUR STACK

- **pytest** `monkeypatch` your port; `respx` / `responses` at HTTP level ↗13
- **Vitest** MSW intercepts HTTP instead of mocking clients ↗15
- **JUnit** Mockito on your interface; WireMock for HTTP ↗18
- **Playwright** `page.route` / HAR for third-party calls ↗29

SMELL CHECK

- **You see** a test that only checks mocks talked to each other → **it means** nothing real is tested → **do** use real collaborators or an integration test
- **You see** mocks of `requests` / `axios` / `RestTemplate` everywhere → **it means** missing port → **do** wrap the client, or fake at the HTTP level

MOCK OR REAL?

DEPENDENCY	IN UNIT TESTS
Your own domain classes	Real
Value objects / DTOs	Real
Database	Fake repository; real in integration
Clock / randomness	Injected fake
Email, SMS, payments	Fake or mock your port
Third-party SDK	Wrap in a port; mock the port

PORTS & ADAPTERS

PART	TESTED BY
Domain logic	Unit tests with fakes of ports
Port (interface)	Contract tests shared by fake and adapter
Adapter (SDK / DB / HTTP)	Integration tests against the real thing
Whole slice	A few end-to-end tests

⚠ COMMON MISTAKES

- Mocking the class under test
- Mocking a library's internals and breaking on every upgrade
- Never running the real adapter anywhere

✓ TIPS & BEST PRACTICES

- When a mock setup gets long, introduce a port
- Run the same contract tests against your fake and the real adapter
- Prefer network-level fakes over client-library mocks

QUICK SUMMARY

- mock at boundaries
- don't mock what you don't own — wrap it
- ports & adapters — fake the port, integration-test the adapter
- over-mocking — tests that lie

Designing for Testability

CODE THAT IS EASY TO TEST IS EASY TO CHANGE

Code is hard to test when it reaches out for its dependencies: calls `now()`, `random()`, `new HttpClient()` or a global. A **seam** is a place where a test can substitute something. The simplest seam is **dependency injection**: pass collaborators in (constructor or parameter). Even better, keep decisions in **pure functions** (same input, same output) and push I/O to a thin outer layer: **functional core, imperative shell**.

NOTES

- Time, randomness, IDs, env vars and I/O are the usual hidden dependencies
- Constructor injection works without a framework
- Pure functions need no doubles at all
- If a test needs heavy patching, the design is asking for a seam

HIDDEN DEPENDENCIES → INJECTED

```
# ✗ hard to test: reaches for time, randomness and the network
function create_invite(email):
    token   = random_hex(16)
    expires = now() + days(7)
    http_post("https://mail.example/send", { to: email, token })
    return Invite(email, token, expires)

# ✓ functional core: pure, no doubles needed
function build_invite(email, token, today) -> Invite:
    return Invite(email, token, expires=today + days(7))

# ✓ imperative shell: dependencies passed in
function create_invite(email, clock, tokens, mailer):
    invite = build_invite(email, tokens.next(), clock.today())
    mailer.send(email, invite.token)
    return invite

test "invite expires in 7 days":
    invite = build_invite("a@x.com", "t1", today="2026-11-04")
    assert invite.expires == "2026-11-11"
```

CODE EXPLAINED

→ The pure `build_invite` is tested with plain values

→ The shell takes a clock, token source and mailer, so tests pass fakes

→ Production wiring passes the real clock and mailer once, at start-up

→ Most logic moves into the easy-to-test core

IN YOUR STACK

- **pytest** pass dependencies in; `monkeypatch` only at the edges ↗13
- **Vitest** `vi.setSystemTime`; pass functions as parameters ↗14
- **JUnit** constructor injection; inject `java.time.Clock` ↗14
- **Playwright** `page.clock` controls time in the browser ↗34

SMELL CHECK

- **You see** tests patching 5 modules to run one function → **it means** hidden dependencies → **do** inject them
- **You see** `static` / global singletons used everywhere → **it means** no seams → **do** pass instances in

HIDDEN DEPENDENCIES

HIDDEN CALL	INJECT INSTEAD
`now() / Date()`	Clock
`random() / uuid()`	Random / IdGenerator
`new HttpClient()`	Client or port
`Global config / env`	Settings object
`File / DB access`	Repository port
`Static singletons`	Instances via constructor

KINDS OF SEAMS

SEAM	EXAMPLE
`Constructor parameter`	Service(repo, clock)
`Method parameter`	invite(email, now)
`Function argument`	retry(fn, sleep=fake_sleep)
`Interface / port`	PaymentGateway
`Module / import`	Patch at import (last resort)

⚠ COMMON MISTAKES

- Making everything an interface "for testing" even when there's one implementation and no I/O
- Injecting the clock but still calling `now()` deep inside
- Refactoring for testability without tests to protect the refactor (see sheet 35)

✓ TIPS & BEST PRACTICES

- Pass a clock wherever business rules depend on time
- Extract decisions into pure functions first; it's the cheapest win
- Wire real dependencies in one place (`main`, the DI container)

QUICK SUMMARY

- seam — where a test can substitute
- dependency injection — pass collaborators in
- inject time, randomness, I/O
- functional core, imperative shell

TDD: Red, Green, Refactor

LET A FAILING TEST DRIVE THE NEXT LINE OF CODE

Test-Driven Development is a short loop: write a small test that **fails** (red), write the **simplest** code that passes (green), then **refactor** with the test as a safety net. Repeat in minutes. Keep a **test list** of cases to cover, take **baby steps**, and use **triangulation** (a second example) to force real logic instead of a hard-coded answer.

NOTES

- Seeing red first proves the test can fail
- "Simplest" can be a hard-coded return; the next test forces generality
- Refactor only on green, test and code both
- TDD is a design technique as much as a testing one

INTERVIEW ANGLE

- ? **Q:** "Do you practice TDD?" Describe red-green-refactor and a real case where it helped
- ? Be honest about where you don't (spikes, UI layout) — interviewers value judgment
- ? Mention that writing the test first shapes a cleaner API

THREE TURNS OF THE LOOP

```
# Test list: 0 items · 1 item · many items · discount · invalid discount

# RED
test "empty cart totals 0":       assert Cart().total() == 0
# GREEN — simplest thing
class Cart: total() -> return 0

# RED
test "one item totals its price":
    cart = Cart(); cart.add(price=5.00)
    assert cart.total() == 5.00
# GREEN — triangulation forces a real implementation
class Cart:
    items = []
    add(price) -> items.append(price)
    total()    -> return sum(items)

# REFACTOR — tests stay green
class Cart:
    lines = []
    add(price, qty=1) -> lines.append(Line(price, qty))
    total()           -> return sum(l.price * l.qty for l in lines)
```

CODE EXPLAINED

- → The first green is a constant; the second test makes the constant wrong
- → Each step is small enough that a failure points straight at the change
- → Refactoring renames and reshapes while the tests prove behavior is unchanged
- → The test list keeps you from forgetting cases while staying in small steps

IN YOUR STACK

- **pytest** `pytest --lf -x` or pytest-watcher for a fast loop ↗15
- **Vitest** `vitest` watch reruns affected tests on save ↗03
- **JUnit** IDE rerun / `gradle test --continuous` ↗03
- **Playwright** UI mode watch for E2E-first work ↗42

SMELL CHECK

- **You see** tests written after all the code, then adjusted until green → **it means** tests confirm code, not requirements → **do** see each test fail first
- **You see** huge steps between greens → **it means** debugging instead of TDD → **do** shrink the next test

THE LOOP

STEP	GOAL	RULE
Red	A failing test for one small behavior	Must fail for the right reason
Green	Pass as fast as possible	Simplest code; no extra features
Refactor	Clean up code and tests	Only on green; behavior unchanged

TECHNIQUES

TECHNIQUE	USE
Baby steps	Smallest test that fails
Fake it	Hard-code, then generalize
Triangulate	Second example forces real logic
Test list	Track cases to cover
Obvious implementation	Skip faking when it's trivial

⚠ COMMON MISTAKES

- Writing several tests before any code
- Skipping the refactor step, so code piles up
- Testing private details because the design came first

✓ TIPS & BEST PRACTICES

- Keep the loop under a few minutes
- Commit on green so you can always step back
- Start a bug fix with a failing regression test

QUICK SUMMARY

- red — failing test first
- green — simplest pass
- refactor — clean on green
- test list + baby steps + triangulation

TDD Styles

TWO SCHOOLS AND TWO DIRECTIONS

Classicist (Chicago / Detroit) TDD uses real objects and checks final state, mocking only awkward boundaries. **Mockist** (London) TDD mocks collaborators and checks interactions, discovering interfaces as it goes. **Inside-out** starts with core domain logic and builds up; **outside-in** starts from a user-facing (often acceptance) test and works inward. Most teams mix them.

NOTES

- Classicist tests survive refactoring better
- Mockist tests give faster design feedback on roles and interfaces
- Outside-in pairs well with BDD acceptance tests
- Spikes and exploratory code are fine without TDD; rewrite with tests afterward

SAME FEATURE, TWO STYLES

```
# CLASSICIST — real collaborators, check state
test "placing an order stores it and reserves stock":
    stock = InMemoryStock({ "A1": 5 })
    orders = InMemoryOrders()
    service = OrderService(orders, stock)
    service.place(cart_with("A1", qty=2))
    assert orders.count() == 1
    assert stock.available("A1") == 3

# MOCKIST — collaborators mocked, check interactions
test "placing an order saves it and reserves stock":
    orders = mock Orders; stock = mock Stock
    service = OrderService(orders, stock)
    service.place(cart_with("A1", qty=2))
    verify stock.reserve("A1", 2)
    verify orders.save(any Order)

# OUTSIDE-IN — start from the outermost failing test
acceptance "customer places an order"   → fails
  → controller test (mocks service)     → fails → passes
  → service test (fakes repositories)   → fails → passes
acceptance test now passes
```

CODE EXPLAINED

→ The classicist test fails only if behavior changes

→ The mockist test also fails if the internal collaboration changes

→ Outside-in keeps every piece of code tied to a real need

→ In-memory fakes give classicist tests mockist-like speed

IN YOUR STACK

- **pytest** fakes in fixtures (classicist) or `mocker` (mockist) ↗14
- **Vitest** `vi.fn` / `vi.mock` for mockist style ↗11
- **JUnit** Mockito-heavy (mockist) vs fakes + slices ↗17
- **Playwright** outside-in: start with a failing E2E test ↗25

SMELL CHECK

You see every refactor breaking dozens of mock-based tests → **it means** over-specified interactions → **do** shift to state checks with fakes

You see no tests above the unit level → **it means** inside-out never reached the outside → **do** add an acceptance test per feature

CLASSICIST VS MOCKIST

ASPECT	CLASSICIST	MOCKIST
Collaborators	Real (fakes at edges)	Mocked
Verifies	State	Interactions
Refactoring	Robust	More brittle
Design feedback	Emerges in refactor	Up front (roles, interfaces)
Typical direction	Inside-out	Outside-in

WHEN TDD PAYS OFF

HELPS	LESS USEFUL
Clear requirements, business rules	Spikes and experiments
Bug fixes (regression first)	Visual layout
Algorithms, parsers	Thin glue code
APIs you're designing	Code you'll throw away

⚠ COMMON MISTAKES

- Treating one school as the only "real" TDD
- Outside-in without ever replacing mocks with integration coverage
- Applying TDD to throwaway spikes and wasting time

✓ TIPS & BEST PRACTICES

- Default to classicist style; mock at boundaries
- Use outside-in for new features with clear user stories
- Throw away spike code and redo it test-first

QUICK SUMMARY

- classicist — real objects, state checks
- mockist — mocks, interaction checks
- outside-in — start from the user
- inside-out — start from the core

BDD & Gherkin

specification by example · Feature / Scenario · Given / When / Then · step definitions · Cucumber · behave · playwright-bdd

SHARED EXAMPLES BECOME TESTS

Behavior-Driven Development starts with conversations about **concrete examples** of how the system should behave, written in plain language everyone can read. **Gherkin** is the common format: a `Feature` with `Scenario`s of `Given` / `When` / `Then` steps. **Step definitions** map each step to code. The value is the shared understanding; the automation is a bonus. Tools: Cucumber (JVM, JS), behave and pytest-bdd (Python), playwright-bdd.

NOTES

- Write scenarios with product and QA before coding ("three amigos")
- Describe behavior, not UI clicks: "When she checks out", not "When she clicks #btn-3"
- Scenario Outlines + Examples tables parameterize scenarios
- Gherkin adds a layer; skip it if nobody outside engineering reads the specs

A FEATURE FILE AND STEPS

```
Feature: Coupons at checkout

  Scenario: A valid coupon reduces the total
    Given a cart with items worth $50.00
    When the shopper applies coupon "SAVE10"
    Then the total is $45.00

  Scenario Outline: Coupons that are not accepted
    Given a cart with items worth $50.00
    When the shopper applies coupon "<code>"
    Then they see the message "<message>"
    Examples:
      | code     | message           |
      | OLD2024  | Coupon expired    |
      | NOPE     | Unknown coupon    |

# step definitions (pseudocode)
given "a cart with items worth ${amount}":  ctx.cart = make_cart(total=amount)
when  "the shopper applies coupon {code}":  ctx.result = apply_coupon(ctx.cart, code)
then  "the total is ${amount}":             assert ctx.cart.total == amount
```

CODE EXPLAINED

- → Anyone on the team can read and challenge the scenarios
- → The Examples table runs the outline once per row
- → Steps are reusable phrases backed by code
- → Steps can drive a domain API, an HTTP API or a browser

IN YOUR STACK

- **pytest** pytest-bdd or behave ↗17
- **Vitest** Cucumber.js / vitest-cucumber; or plain `describe` with GWT names ↗04
- **JUnit** Cucumber-JVM on the JUnit Platform ↗01
- **Playwright** playwright-bdd generates tests from `.feature` files ↗61

SMELL CHECK

- **You see** scenarios full of CSS selectors and clicks → **it means** UI scripts, not behavior → **do** write steps in domain language
- **You see** feature files nobody outside engineering reads → **it means** overhead without the benefit → **do** use plain tests with GWT names

GHERKIN KEYWORDS

KEYWORD	PURPOSE
Feature	The capability
Scenario	One example
Given	Starting context
When	The action
Then	Expected outcome
And / But	More of the previous step
Background	Steps before every scenario
Scenario Outline + Examples	Parameterized scenario

BDD TOOLS

STACK	TOOL
Code review of tests	Mystery guests, logic, eager tests
Lint rules	Common anti-patterns
Random order runs	Shared state
Slowest-tests report	Slow tests
Mutation testing	Weak or missing assertions

⚠ COMMON MISTAKES

- Writing Gherkin alone, after the code, as a reporting format
- Huge step libraries with near-duplicate phrases
- One scenario testing a whole user journey with 30 steps

✓ TIPS & BEST PRACTICES

- Keep scenarios to a handful of steps
- Run example-mapping sessions before writing scenarios
- Use BDD where business rules are rich and stakeholders care

QUICK SUMMARY

- BDD — shared concrete examples
- Gherkin — Feature / Scenario / Given-When-Then
- step definitions — phrases → code
- domain language, not UI clicks

Integration Testing Basics

PROVE THE PIECES FIT

An **integration test** runs several parts of the system together, usually with **real infrastructure**: a real database, message broker, or HTTP stack. It catches what unit tests can't: wrong SQL, mismatched serialization, missing configuration, transaction bugs. The trade-offs are speed and **isolation**: each test must start from a known state and clean up.

NOTES

- Use the same database engine as production (containers make this easy)
- In-memory stand-ins (H2, SQLite for Postgres) hide dialect bugs
- Keep a separate, slower job for integration tests if needed
- Start the app once per suite; reset data per test

AN INTEGRATION TEST WITH REAL INFRASTRUCTURE

```
suite setup (once):
    db  = start_container("postgres:17")
    app = start_app(database_url=db.url)

each test setup:
    db.begin_transaction()            # or truncate tables

test "placing an order writes order and lines":
    response = app.post("/orders", { sku: "A1", qty: 2 })
    assert response.status == 201
    row = db.query_one("select status, total from orders where id = ?", response.body.id)
    assert row == ("NEW", 20.00)

each test teardown:
    db.rollback()

suite teardown:
    app.stop(); db.stop()
```

CODE EXPLAINED

→ Expensive things (container, app) start once per suite

→ Each test runs in a transaction rolled back afterward, so tests stay independent

→ The test checks the database row, proving the SQL and mapping really work

→ The same pattern works for queues, caches and search engines

IN YOUR STACK

- **pytest** session-scoped fixtures for the DB, function-scoped rollback ↗07
- **Vitest** `globalSetup` starts services; per-test reset in hooks ↗05
- **JUnit** `@SpringBootTest` + Testcontainers ↗19
- **Playwright** `webServer` starts the app before E2E tests ↗49

SMELL CHECK

⚑ **You see** integration tests that pass on H2 but fail on Postgres → **it means** the stand-in lies → **do** test on the real engine in a container

⚑ **You see** tests that only pass in a fixed order → **it means** leftover data → **do** reset state per test

WHAT INTEGRATION TESTS CATCH

BUG	EXAMPLE
Wrong SQL / query	Missing join condition
Mapping / serialization	Field renamed in JSON
Transactions	Partial write on failure
Configuration	Wrong connection settings
Migrations	Column missing in real schema

ISOLATION STRATEGIES

STRATEGY	TRADE-OFF
Transaction rollback	Fast; can't test commits or multiple connections
Truncate tables	Real commits; slower
Fresh schema per test	Strong isolation; slowest
Unique data per test	Parallel-safe; needs discipline

⚠ COMMON MISTAKES

- Calling everything with a database an "integration test" but mocking the database
- Sharing one database between parallel test workers without unique data
- No integration tests at all, trusting mocks completely

✓ TIPS & BEST PRACTICES

- Test each adapter (repository, HTTP client, queue) against the real thing at least once
- Keep integration tests in their own folder / tag
- Measure suite time and set a budget

QUICK SUMMARY

- integration — real pieces + real infra
- same engine as production
- start once, reset per test
- catches SQL, config and mapping bugs

Databases in Tests

REAL DATABASES WITHOUT SLOW, SHARED MESS

Test against the **same engine and version** as production, started in a container. Apply the real **migrations** once, so tests also prove migrations work. Each test then gets a clean slate by **rolling back** a transaction, **truncating** tables, or using **unique data**. Seed only shared reference data; each test creates the rest itself.

NOTES

- Container start-up costs seconds; do it once per run
- Rollback can't test code that commits or uses multiple connections
- Parallel workers need separate databases or schemas
- Never point tests at a shared dev or staging database

INTERVIEW ANGLE

- ? **Q:** "How do you test code that uses a database?" Real engine in a container, migrations once, isolation per test
- ? Compare with in-memory fakes: faster, but miss SQL dialect bugs
- ? Mention parallel safety (database per worker or unique data)

DATABASE TEST LIFECYCLE

```
once per run:
    pg = PostgresContainer("postgres:17").start()
    run_migrations(pg.url)                  # the real migration files
    seed(pg, reference_data="countries, plans")

per worker (parallel runs):
    schema = "test_" + worker_id            # or one database per worker

per test:
    tx = pg.begin()
    yield                                   # test runs inside tx
    tx.rollback()

test "monthly report counts only paid orders":
    make_order(status="PAID", month="2026-10")
    make_order(status="NEW",  month="2026-10")
    assert monthly_report("2026-10").order_count == 1
```

CODE EXPLAINED

- → Running migrations in tests catches broken migrations before deploy
- → Reference data is seeded once; scenario data is created per test
- → A schema per worker keeps parallel tests from colliding
- → Factories (sheet 08) create rows with valid defaults

IN YOUR STACK

- **pytest** pytest-django `django_db`; session DB fixture + rollback ↗21
- **Vitest** Testcontainers for Node in `globalSetup` ↗05
- **JUnit** Testcontainers + `@ServiceConnection`; `@DataJpaTest` ↗21
- **Playwright** global setup seeds data; API calls create test records ↗26

SMELL CHECK

- **You see** tests that fail when run twice → **it means** data left behind → **do** roll back or clean up per test
- **You see** a 2 GB fixture dump loaded before tests → **it means** mystery guests everywhere → **do** seed minimal reference data, build the rest per test

CLEANUP STRATEGIES

STRATEGY	SPEED	LIMITS
`Rollback per test`	Fastest	No commits, one connection
`Truncate tables`	Fast	Real commits; reset sequences
`Recreate schema`	Slow	Strongest isolation
`Unique data`	Fast	Parallel-safe; data accumulates

TOOLS BY STACK

STACK	DATABASE TESTING
`Python`	pytest-django, testcontainers-python, SQLAlchemy sessions
`Node`	Testcontainers, Prisma test DBs
`Java`	Testcontainers, @DataJpaTest, Flyway / Liquibase
`Any`	Docker Compose test profile

⚠ COMMON MISTAKES

- Using a shared staging database for automated tests
- Skipping migrations and creating schema from ORM models, so migrations go untested
- Huge shared fixture files that every test depends on

✓ TIPS & BEST PRACTICES

- Pin the database image version to production's
- Use a container reuse option locally to avoid restart cost
- Assert with SQL when you need to prove what was actually stored

QUICK SUMMARY

- real engine in a container
- migrations once per run
- rollback / truncate / unique data per test
- database per worker for parallel runs

Testing HTTP APIs

TEST THE CONTRACT YOUR CLIENTS USE

An API test sends a request and checks **status**, **headers** and **body**, the parts clients depend on. Test the **error paths** as carefully as success: validation errors, missing auth, wrong role, not found, conflicts. In-process **test clients** call the app without a network and are fast; real HTTP against a running server catches proxy, TLS and serialization issues. The HTTP set covers status codes and headers.

NOTES

- Validate responses against the OpenAPI schema to catch drift
- Create a token / session once per test file, not per request
- Check `Content-Type`, `Location` and cache headers, not just status
- Test pagination edges: empty page, last page, invalid cursor

SUCCESS AND FAILURE PATHS

```
test "POST /orders creates an order":
    res = client.post("/orders", json={ sku: "A1", qty: 2 }, auth=token("ann"))
    assert res.status == 201
    assert res.headers["Location"] == "/orders/" + res.body.id
    assert res.body matches schema("Order")

test "POST /orders without a token is 401":
    assert client.post("/orders", json={…}).status == 401

test "POST /orders by a read-only user is 403":
    assert client.post("/orders", json={…}, auth=token("viewer")).status == 403

test "invalid quantity returns a problem+json error":
    res = client.post("/orders", json={ sku: "A1", qty: 0 }, auth=token("ann"))
    assert res.status == 422
    assert res.headers["Content-Type"] == "application/problem+json"
    assert res.body.errors == [{ field: "qty", message: "must be > 0" }]

test "GET another user's order is 404, not 403":
    assert client.get("/orders/" + bob_order, auth=token("ann")).status == 404
```

CODE EXPLAINED

- → Status, `Location` header and schema are all part of the contract
- → 401 (who are you?) vs 403 (not allowed) vs 404 (hide existence) each get a test
- → Validation errors check the machine-readable error body
- → Schema validation catches fields added or renamed by accident

IN YOUR STACK

- **pytest** FastAPI `TestClient`, Django `client`, DRF `APIClient` ↗21
- **Vitest** Supertest or `fetch` against the app; MSW for outbound calls ↗15
- **JUnit** `MockMvcTester` / `WebTestClient` ↗20
- **Playwright** `request` fixture for API tests ↗31

SMELL CHECK

- **You see** only happy-path API tests → **it means** auth and validation bugs go unnoticed → **do** add a test per error status
- **You see** assertions only on status codes → **it means** body and header regressions slip through → **do** validate the body against the schema

PER-ENDPOINT CHECKLIST

CHECK	EXAMPLE STATUS
Success	200 / 201 / 204
Validation error	400 / 422
No auth	401
Wrong role	403
Not found / not yours	404
Conflict	409
Response schema	Matches OpenAPI

TEST CLIENT VS REAL HTTP

APPROACH	PROS	CONS
In-process client	Fast, no ports	Skips network stack
Real HTTP to local server	Real serialization, middleware	Slower, ports
Against deployed env	Real infra (smoke)	Slow, shared data

⚠ COMMON MISTAKES

- Asserting on the exact error message text that product changes often
- Reusing one admin token for every test and missing authorization bugs
- Testing through the UI what an API test could cover faster

✓ TIPS & BEST PRACTICES

- Generate request / response types from the OpenAPI spec and reuse them in tests
- Keep one test per status code an endpoint can return
- Run a small API smoke suite against each deployed environment

QUICK SUMMARY

- status + headers + body = the contract
- test every error path (400 / 401 / 403 / 404 / 409 / 422)
- schema validation catches drift
- test client fast · real HTTP thorough

Contract Testing

SERVICES AGREE WITHOUT RUNNING TOGETHER

When services talk over APIs, a change on one side can break the other. **Contract tests** check that both sides agree on the API **without** deploying them together. In **consumer-driven** contracts (Pact), the consumer's tests record the requests it makes and the responses it needs; the **provider** then replays them in its own CI to verify it still satisfies every consumer. A **broker** shares contracts and results.

NOTES

- Contracts test the **shape** of interactions, not business logic
- Each consumer records only the fields it uses, so providers can add fields freely
- "Can I deploy?" checks with the broker gate releases
- Schema-based alternatives compare against an OpenAPI spec

INTERVIEW ANGLE

- ? **Q:** "What is contract testing?" Consumer records expectations; provider verifies them in its own pipeline
- ? Why not just E2E? Faster, pinpoints which side broke, no shared environment needed
- ? Mention Pact and a broker by name

CONSUMER TEST → CONTRACT → PROVIDER VERIFICATION

```
# CONSUMER (orders-ui) — runs against a mock provider
contract "orders-ui" ↔ "orders-api":
    given "order 7 exists"
    upon receiving GET /orders/7
    respond with 200 {
        id:     like(7),            # any number
        status: term("NEW|PAID", example="PAID"),
        total:  like(45.00)
    }
test "order page shows the status":
    order = OrdersClient(mock_provider.url).get(7)
    assert order.status == "PAID"
# → writes contract file, published to the broker

# PROVIDER (orders-api) — in its own CI
verify contracts from broker for "orders-api":
    state "order 7 exists": db.insert(Order(id=7, status="PAID"))
    replay each request against the real API, compare responses
# → results published; deploy gated on "can-i-deploy"
```

CODE EXPLAINED

- → Matchers like `like()` check type, not exact values
- → Provider states set up data so each interaction can be replayed
- → If the provider renames `status`, its CI fails before release
- → Both sides deploy independently once verification passes

IN YOUR STACK

- **pytest** pact-python (consumer and provider) ↗17
- **Vitest** @pact-foundation/pact with MSW-style mock server ↗15
- **JUnit** Pact JVM JUnit 5 extension, Spring Cloud Contract ↗14
- **Playwright** not a contract tool; API tests can validate against OpenAPI ↗31

SMELL CHECK

- **You see** a shared staging environment where teams wait to run E2E suites → **it means** integration is checked too late → **do** add contract tests per service pair
- **You see** contracts that pin every response field exactly → **it means** brittle contracts → **do** use type matchers and only fields the consumer uses

CONTRACT VS E2E

ASPECT	CONTRACT TESTS	E2E TESTS
Needs all services running	No	Yes
Speed	Seconds	Minutes
Pinpoints the broken side	Yes	Rarely
Tests business flows	No	Yes
Runs in	Each service's CI	Shared environment

CONTRACT STYLES

STYLE	HOW
Consumer-driven (Pact)	Consumers record, provider verifies
Provider-driven / schema	Compare against OpenAPI spec
Spring Cloud Contract	Contracts in a DSL, stubs generated

⚠ COMMON MISTAKES

- Using contract tests to test business logic
- Consumers asserting fields they don't use
- Never verifying contracts on the provider side

✓ TIPS & BEST PRACTICES

- Start with your most-changed API between two teams
- Wire "can-i-deploy" into release pipelines
- Keep a few E2E tests for critical journeys on top

QUICK SUMMARY

- contract — agreed API shape
- consumer-driven — consumer records needs
- provider verifies in its own CI
- broker + can-i-deploy — safe releases

Component Tests vs End-to-End Tests

TEST UI AT THE LOWEST LEVEL THAT WORKS

A **component test** renders one UI component (or a small group) in isolation with fake data and checks what a user sees and can do. An **end-to-end** test drives the real app in a real browser. Component tests are faster and pinpoint problems; E2E tests prove the whole thing works together. Both should interact like a user: find elements by **role** and **label**, not by CSS classes.

NOTES

- "The more your tests resemble the way your software is used, the more confidence they give" (Testing Library)
- Mock the network at the component level; use the real backend in E2E
- Visual and layout checks need a real browser
- Querying by role doubles as an accessibility check

SAME FORM, TWO LEVELS

```
# COMPONENT TEST — one component, fake data, fast
test "login form shows an error for a wrong password":
    render(LoginForm, onSubmit=fake returning error("wrong password"))
    type(by_label("Email"), "ann@x.com")
    type(by_label("Password"), "nope")
    click(by_role("button", name="Sign in"))
    expect(by_role("alert")).has_text("Wrong password")

# END-TO-END — real app, real backend, real browser
test "user can sign in and see their orders":
    open_page("/login")
    fill(by_label("Email"), "ann@x.com")
    fill(by_label("Password"), test_password)
    click(by_role("button", name="Sign in"))
    expect(by_role("heading", name="Your orders")).visible
```

CODE EXPLAINED

- → The component test covers the error state without a server
- → The E2E test covers the real login flow once
- → Both find elements by label and role, as a screen reader would
- → Edge cases (lockout, validation) belong in component or API tests

IN YOUR STACK

- **pytest** server-rendered views via Django `client`; UI via Playwright ↗21
- **Vitest** Testing Library + jsdom, or browser mode ↗18
- **JUnit** `@WebMvcTest` for server views; UI via Playwright ↗20
- **Playwright** component testing or full E2E ↗53

SMELL CHECK

- **You see** selectors like `.btn-primary > span:nth-child(2)` → **it means** tests coupled to markup → **do** query by role, label or text
- **You see** every UI state tested end-to-end → **it means** slow, flaky suite → **do** move states into component tests

COMPONENT VS E2E

ASPECT	COMPONENT	E2E
Runs	jsdom or browser, one component	Real browser, whole app
Backend	Faked	Real (or staging)
Speed	ms–1 s	Seconds
Catches	Rendering, states, events	Wiring, routing, auth, real data
Count	Many	Few

QUERY PRIORITY

PREFER	OVER
`by role + name`	by CSS class
`by label`	by placeholder
`by visible text`	by DOM position
`by test ID`	by XPath

⚠ COMMON MISTAKES

- Snapshotting whole component trees instead of asserting behavior
- Testing implementation details like component state or hooks directly
- No E2E test at all for the main flows

✓ TIPS & BEST PRACTICES

- Cover every visual state of a component (loading, empty, error, full)
- If an element can't be found by role, fix its accessibility
- Keep E2E for journeys that cross pages or services

QUICK SUMMARY

- component — one piece, fake data, fast
- E2E — whole app, real browser
- query by role / label, like a user
- states in component tests, journeys in E2E

End-to-End Test Strategy

FEW, VALUABLE, RELIABLE

E2E tests are the most expensive tests you have, so spend them on **critical user journeys**: sign up, log in, search, buy, the flows that lose money or users when broken. Make each test independent: set up **data and auth through APIs** rather than clicking through the UI, use **stable selectors** (roles, test IDs), and rely on auto-waiting instead of sleeps. Run them against a **known environment**.

NOTES

- Reuse a logged-in session (storage state) instead of logging in every test
- Create test data via API or seed scripts; clean up or use unique names
- Run a small smoke subset on every deploy, the full set before release
- Record traces / videos on failure (sheet 31)

A JOURNEY TEST WITH API SETUP

```
suite setup:
    api.login_as("ann") → save session to "ann.json"      # once

test "returning customer reorders a past purchase" [tag: critical]:
    use session "ann.json"                                  # skip the login UI
    order = api.create_order(customer="ann", sku="A1")     # fast setup via API
    open_page("/orders")
    click(by_role("row", name=order.reference) → by_role("button", name="Reorder"))
    click(by_role("button", name="Checkout"))
    expect(by_role("heading", name="Order confirmed")).visible

# ✗ avoid
sleep(3)                                  # use waiting assertions
click("#app > div:nth-child(4) > button")  # use roles / test IDs
test 2 depends on data from test 1        # each test sets up its own
```

CODE EXPLAINED

→ Logging in once and reusing the session removes minutes from a suite

→ API setup makes the test about the journey, not the setup screens

→ Waiting assertions retry until the page is ready, so no sleeps

→ Tagging lets CI run critical journeys on every deploy

IN YOUR STACK

- **pytest** `pytest-playwright` for Python E2E
- **Vitest** browser mode for components; Playwright for journeys ↗19
- **JUnit** Playwright for Java with JUnit
- **Playwright** storage-state auth, API setup, tags ↗27

SMELL CHECK

- **You see** an E2E suite that takes an hour → **it means** too many tests or UI setup → **do** cut to critical journeys and set up via API
- **You see** `sleep` calls in E2E tests → **it means** timing guesses → **do** use auto-waiting assertions

CHOOSING E2E TESTS

INCLUDE	PUSH DOWN INSTEAD
Sign up / log in	Password rules → unit
Search → buy	Price edge cases → unit
Core create / edit flow	Validation messages → component
Payment happy path	Payment errors → API / integration
Cross-service journey	Each service's rules → its own tests

WHERE E2E TESTS RUN

ENVIRONMENT	USE
Local app + test DB	Development, PR checks
Preview / ephemeral env	PR checks with real infra
Staging	Pre-release full run
Production	Smoke only, safe read paths

⚠ COMMON MISTAKES

- Logging in through the UI in every test
- Tests that share and mutate the same records
- Running the full E2E suite against production

✓ TIPS & BEST PRACTICES

- Keep a short list of critical journeys and review it each quarter
- Give every test unique data (timestamped names)
- Fail fast in CI with a smoke subset first

QUICK SUMMARY

- critical journeys only
- auth and data via API, not the UI
- roles / test IDs + auto-waiting
- independent tests, known environment

Flaky Tests

TESTS THAT PASS AND FAIL WITHOUT A CODE CHANGE

A **flaky test** gives different results on the same code. Flakes destroy trust: people rerun CI until green and stop reading failures. Common causes: **timing** (sleeps, races, animations), **order dependence** and **shared state**, **real network or time**, and **resource limits** in CI. Detect them by rerunning and randomizing order, **quarantine** known flakes so they don't block others, then **fix or delete** them.

NOTES

- Retries hide flakes; if you use them, report retried tests
- Run a suspect test many times in a loop to reproduce
- Randomized order exposes hidden dependencies
- Quarantine needs an owner and a deadline

INTERVIEW ANGLE

- ? **Q:** "How do you handle flaky tests?" Find the root cause; quarantine with an owner; never just add retries
- ? List causes: timing, order, shared state, network, time zones
- ? Mention tooling: rerun loops, random order, CI flake reports

FINDING AND FIXING A FLAKE

```
# 1. reproduce
run test "search shows results" 200 times → 3 failures

# 2. look for the cause
failure trace: clicked "Search" before the input was enabled
# cause: timing — test acted before the page was ready

# 3. fix the cause, not the symptom
# ✗ sleep(2) then click
# ✓ wait for the condition
expect(by_role("textbox", name="Search")).enabled
click(by_role("button", name="Search"))

# order dependence: passes alone, fails in the suite
run suite with random order, seed=8812 → fails
# cause: test A leaves a feature flag on; test B assumes off
# fix: reset flags in per-test setup

# quarantine while fixing
mark test "imports nightly file" as quarantined(owner="bob", issue=#311, until="2026-11-30")
```

CODE EXPLAINED

- → Running a test hundreds of times turns "sometimes" into evidence
- → Traces and screenshots show what the page looked like at failure
- → The real fix waits for a condition instead of a fixed time
- → A fixed random seed reproduces an order-dependent failure

IN YOUR STACK

- **pytest** pytest-randomly, `--count`, pytest-rerunfailures (temporary) ↗19
- **Vitest** `--sequence.shuffle`, `retry`, repeats ↗21
- **JUnit** `@RepeatedTest`, random method order ↗13
- **Playwright** `--repeat-each`, `retries`, `--fail-on-flaky-tests` ↗39

SMELL CHECK

- **You see** a global "retry 3 times" setting → **it means** flakes are hidden, not fixed → **do** report retried tests and fix them
- **You see** "just rerun CI" in team chat → **it means** red no longer means broken → **do** track flake rate and quarantine with owners

CAUSES AND FIXES

CAUSE	FIX
Fixed sleeps / races	Wait for conditions; await properly
Order dependence	Fresh state per test; random order
Shared data / files / ports	Unique per test or worker
Real time / time zones	Fixed clock; explicit zone
Real network	Fake at the boundary
CI resource limits	Fewer workers; longer timeouts only as last resort
Animations / async UI	Disable animations; web-first assertions

FLAKE POLICY

STEP	RULE
Detect	Track retried / intermittent tests in CI
Quarantine	Skip in required job, with owner + issue
Fix	Root cause, not a longer timeout
Deadline	Fix or delete within a set time
Report	Flake rate per week

⚠ COMMON MISTAKES

- Adding sleeps to "fix" timing flakes
- Raising timeouts until the flake goes away
- Quarantining tests forever with no owner

✓ TIPS & BEST PRACTICES

- Track a flake rate per test in CI
- Reproduce with the same seed, worker count and machine size as CI
- Delete a flaky test that tests nothing important; it's cheaper than living with it

QUICK SUMMARY

- flaky — same code, different results
- causes — timing, order, shared state, network, time
- quarantine with owner + deadline
- retries hide; root cause fixes

Property-Based Testing

STATE A RULE, LET THE TOOL FIND COUNTEREXAMPLES

Example tests check a few hand-picked inputs. **Property-based tests** state a rule that should hold for **all** inputs (a **property**), and a library **generates** hundreds of inputs to try to break it. When it finds a failure, it **shrinks** the input to the smallest case that still fails. Good properties: round-trips (`decode(encode(x)) == x`), invariants (total never negative), and agreement with a simple reference implementation.

NOTES

- Properties complement examples; keep a few examples for readability
- Failing inputs are saved and replayed on the next run
- Avoid re-implementing the function in the test
- Great for parsers, serializers, math, data structures

PROPERTIES INSTEAD OF EXAMPLES

```
# round-trip
property "decode reverses encode":
    for all s in strings():
        assert decode(encode(s)) == s

# invariant
property "cart total is never negative":
    for all prices in lists(floats(min=0, max=1e6)), d in floats(0, 0.99):
        assert cart_total(prices, discount=d) >= 0

# idempotence
property "normalizing an email twice changes nothing":
    for all e in emails():
        assert normalize(normalize(e)) == normalize(e)

# agreement with a simple reference (oracle)
property "fast_sort agrees with built-in sort":
    for all xs in lists(integers()):
        assert fast_sort(xs) == sorted(xs)

# failure output after shrinking:
#   Falsifying example: s = "\x00"    (minimal input)
```

CODE EXPLAINED

- → A round-trip test catches encoding bugs no one thought to try
- → Shrinking turns a 400-character random string into the one character that breaks it
- → Idempotence and invariants describe behavior without computing exact outputs
- → A slow but obviously correct reference checks a fast implementation

IN YOUR STACK

- **pytest** Hypothesis `@given(st.lists(st.floats()))` ↗22
- **Vitest** fast-check `fc.assert(fc.property(...))` ↗06
- **JUnit** jqwik `@Property` + `@ForAll` on the JUnit Platform ↗11
- **Playwright** not a fit; use it for unit-level logic

SMELL CHECK

- **You see** a property that re-computes the expected value the same way the code does → **it means** the test copies the bug → **do** use a different property (round-trip, invariant, oracle)
- **You see** generators restricted until nothing interesting is generated → **it means** the property is too strict → **do** widen inputs and handle edge cases explicitly

PROPERTY PATTERNS

PATTERN	EXAMPLE
Round-trip	parse(format(x)) == x
Invariant	len(sort(xs)) == len(xs)
Idempotence	f(f(x)) == f(x)
Oracle	fast(x) == slow_but_simple(x)
Commutativity	merge(a, b) == merge(b, a)
Never crashes	parse(any_bytes) raises only ParseError

LIBRARIES

LANGUAGE	LIBRARY
Python	Hypothesis
JavaScript / TS	fast-check
Java	jqwik
Haskell	QuickCheck (the original)
Scala	ScalaCheck
Rust	proptest

⚠ COMMON MISTAKES

- Writing properties that are just the implementation again
- Ignoring the saved failing example and "fixing" by narrowing generators
- Using property tests for code with side effects and no clear rule

✓ TIPS & BEST PRACTICES

- Start with round-trip properties for any serializer or parser
- Keep the generator close to real input shapes
- Turn each shrunk failure into a permanent example test

QUICK SUMMARY

- property — a rule for all inputs
- generators — many inputs
- shrinking — minimal failing case
- round-trip · invariant · idempotence · oracle

Snapshot & Approval Testing

COMPARE AGAINST A REVIEWED OUTPUT

A **snapshot** (or **approval**) test serializes an output and compares it with a stored, reviewed copy. It's cheap to write for large outputs: rendered HTML, reports, generated code, CLI text. Its weakness is review: when output changes, someone must read the diff before **accepting** it, or the test turns into "update until green". Keep snapshots small and stable.

SNAPSHOT WORKFLOW

```
test "invoice renders":
    html = render_invoice(sample_order())
    assert html matches snapshot          # first run: writes invoice.snap

# later, someone changes the template →
#   FAIL invoice renders
#   - <td>Total</td><td>$45.00</td>
#   + <td>Total due</td><td>$45.00</td>
# reviewer reads the diff → intended? accept with --update

test "order JSON shape":
    data = to_json(make_order())
    data = normalize(data, id="<id>", created_at="<date>")
    assert data matches snapshot

test "CLI help text":
    assert run_cli(["--help"]).stdout matches inline_snapshot("""
        Usage: orders [OPTIONS] COMMAND
        ...
    """)
```

NOTES

- Commit snapshot files and review their diffs in PRs
- Normalize volatile values (dates, IDs) before snapshotting
- Inline snapshots keep small outputs in the test itself
- Golden master testing pins legacy behavior (sheet 35)

CODE EXPLAINED

→ The diff shows exactly what changed in the output

→ Accepting rewrites the stored snapshot; that's a deliberate, reviewed act

→ Normalizing volatile fields keeps snapshots stable

→ Inline snapshots are reviewed in the test file itself

IN YOUR STACK

- **pytest** syrupy `snapshot` fixture ↗22
- **Vitest** `toMatchSnapshot`, `toMatchInlineSnapshot`, `toMatchFileSnapshot` ↗16
- **JUnit** ApprovalTests.Java, or JSON / golden-file comparisons ↗13
- **Playwright** `toHaveScreenshot`, `toMatchAriaSnapshot` ↗35

SMELL CHECK

- **You see** PRs that update 40 snapshot files with no comment → **it means** nobody reviews them → **do** keep snapshots small and require diff review
- **You see** snapshots failing every run on timestamps → **it means** volatile data → **do** normalize or freeze it

GOOD AND BAD FITS

GOOD FIT	POOR FIT
Rendered HTML / emails	Pages with dynamic content
Serializer / formatter output	Whole component trees
CLI help text	Output with timestamps
Generated code / SQL	Anything nobody will review

VARIANTS

VARIANT	STORES
File snapshot	Separate .snap file
Inline snapshot	Inside the test
Approval test	approved vs received files
Visual snapshot	Screenshot image
Golden master	Outputs for many recorded inputs

⚠ COMMON MISTAKES

- Updating snapshots to make CI green without reading the diff
- Snapshotting entire pages or component trees
- Snapshots containing secrets or personal data

✓ TIPS & BEST PRACTICES

- Prefer explicit assertions for the few facts that matter
- Review snapshot diffs like code
- Delete obsolete snapshots regularly

QUICK SUMMARY

- snapshot — stored, reviewed output
- diff review is the real test
- normalize volatile values
- small and stable beats big and noisy

Mutation Testing

TESTING YOUR TESTS

Mutation testing makes small changes to your code (**mutants**): flips > to >=, deletes a line, replaces `true` with `false`. Then it runs your tests. If a test fails, the mutant is **killed**; if all pass, it **survived**, which means no test notices that change. The **mutation score** (killed ÷ total) measures how well tests check behavior, which coverage can't.

NOTES

- Expensive: the suite runs once per mutant; scope it to changed code
- Some surviving mutants are equivalent (no behavior change); ignore those
- Incremental modes reuse previous results
- Read survivors to find missing assertions

INTERVIEW ANGLE

- **Q:** "Is 100% coverage a good goal?" — no; coverage shows lines ran, not that tests check them
- Mutation score is a better signal of test strength
- Example: a test that calls a function but asserts nothing gives 100% coverage and kills no mutants

A SURVIVING MUTANT

```
function shipping_fee(total):
    if total >= 50.00: return 0.00
    return 5.99

test "orders over $50 ship free":   assert shipping_fee(80.00) == 0.00
test "small orders pay shipping":   assert shipping_fee(10.00) == 5.99
# coverage: 100%

# mutant 1:  total >= 50.00  →  total > 50.00
#   both tests still pass  → SURVIVED
# mutant 2:  return 5.99  →  return 0.00
#   "small orders" fails   → KILLED

# fix: test the boundary
test "exactly $50 ships free":      assert shipping_fee(50.00) == 0.00
# mutant 1 now KILLED

# mutation score: 2 killed / 2 = 100%
```

CODE EXPLAINED

- → Line coverage was already 100% but the boundary wasn't tested
- → The surviving mutant points to the exact missing test
- → Boundary values are where most survivors hide
- → Run it on changed files in PRs, the whole code base occasionally

IN YOUR STACK

- **pytest** mutmut or cosmic-ray ↗18
- **Vitest** StrykerJS with the Vitest runner ↗20
- **JUnit** PIT (pitest) via Maven / Gradle plugin ↗23
- **Playwright** not a fit for E2E; mutate unit-tested logic

SMELL CHECK

- **You see** high coverage but many surviving mutants → **it means** tests run code without checking results → **do** add assertions on outputs and boundaries
- **You see** mutation runs taking hours in every PR → **it means** no scoping → **do** run incrementally on changed files

COMMON MUTATORS

MUTATOR	EXAMPLE
Conditional boundary	>= → >
Negate condition	== → !=
Math	+ → -
Return value	return x → return null / 0
Remove call	delete a void call
Boolean	true → false

RESULTS

RESULT	MEANS
Killed	A test failed: good
Survived	No test noticed: missing check
No coverage	No test ran that code
Timeout	Mutant caused a hang (counts as killed)
Equivalent	Change doesn't alter behavior: ignore

⚠ COMMON MISTAKES

- Chasing 100% mutation score including equivalent mutants
- Running full mutation testing on every commit
- Ignoring survivors because coverage looks fine

✓ TIPS & BEST PRACTICES

- Start with your most critical module
- Use survivors as a to-do list of missing assertions
- Combine with coverage: coverage finds untested code, mutation finds weakly tested code

QUICK SUMMARY

- mutant — small code change
- killed = a test noticed · survived = none did
- mutation score — test strength
- scope to changed code; it's expensive

Fuzz Testing

THROW UNEXPECTED INPUT UNTIL SOMETHING BREAKS

Fuzzing feeds huge numbers of malformed or random inputs to code that parses or handles untrusted data, looking for crashes, hangs, memory errors and unexpected exceptions. **Coverage-guided** fuzzers (libFuzzer, AFL, Jazzer, Atheris, Go's native fuzzing) watch which code paths each input reaches and mutate the inputs that find new paths. A saved **corpus** of interesting inputs makes later runs smarter.

NOTES

- Best targets: parsers, decoders, file formats, protocol handlers
- Every crash found becomes a regression test
- Run continuously (OSS-Fuzz, CI nightly) rather than once
- Related to property-based testing; fuzzers focus on crashes and security

A FUZZ TARGET

```
fuzz_target(data: bytes):
    try:
        doc = parse_invoice(data)
    except InvoiceParseError:
        return                      # expected: bad input rejected cleanly
    # anything else (crash, other exception, hang) is a bug
    assert doc.total >= 0

# run
fuzz parse_invoice --corpus corpus/invoices --max-time 10m
# found: IndexError with input b"INV\x00\x00;;"
# → saved to crashes/crash-3f2a…
# → add as a regression test:
test "parser rejects truncated header":
    raises InvoiceParseError: parse_invoice(b"INV\x00\x00;;")
```

CODE EXPLAINED

→ The target defines what "acceptable" failure looks like

→ Any other exception is reported with the input that caused it

→ The corpus starts with real sample files and grows with interesting inputs

→ Each finding becomes a normal example test

IN YOUR STACK

- **pytest** Atheris (coverage-guided) or Hypothesis for structured fuzzing ↗22
- **Vitest** Jazzer.js; fast-check for structured input ↗06
- **JUnit** Jazzer with `@FuzzTest` on JUnit ↗14
- **Playwright** not a fuzzer; API tests can send malformed payloads

SMELL CHECK

⚑ **You see** a parser of untrusted files with only happy-path tests → **it means** security bugs waiting → **do** add a fuzz target and run it nightly

⚑ **You see** fuzz crashes triaged once and forgotten → **it means** regressions return → **do** turn each crash into a test

FUZZING APPROACHES

APPROACH	HOW
Random / dumb	Random bytes; finds shallow bugs
Mutation-based	Mutates sample inputs
Coverage-guided	Keeps inputs that reach new code
Structure-aware	Generates valid-ish formats (grammars)

TOOLS

LANGUAGE	FUZZER
Go	go test -fuzz (built in)
Python	Atheris
Java / JVM	Jazzer
JavaScript	Jazzer.js
C / C++ / Rust	libFuzzer, AFL++, cargo-fuzz
Continuous	OSS-Fuzz, ClusterFuzzLite

⚠ COMMON MISTAKES

- Fuzzing code that has no clear failure condition
- Running a fuzzer once for a minute and calling it done
- Not saving the corpus between runs

✓ TIPS & BEST PRACTICES

- Seed the corpus with real sample inputs
- Run fuzzers nightly with a time limit
- Fuzz anything that parses data from users or the network

QUICK SUMMARY

- fuzzing — massive unexpected input
- coverage-guided — mutate inputs that find new paths
- corpus — remembered interesting inputs
- crash → regression test

Code Coverage

line · branch · condition · function coverage · thresholds · coverage on changed code · what coverage misses

WHICH CODE RAN — NOT WHETHER IT WAS CHECKED

Coverage tools record which code executed during the tests. **Line** coverage counts executed lines; **branch** coverage checks both sides of every `if`; **condition** coverage checks each part of a compound condition. Coverage is great at showing **untested** code, and weak at proving tested code is **checked**: a test with no assertions still adds coverage. Use thresholds as a floor, and watch coverage on **changed** code.

NOTES

- Branch coverage is stricter and more useful than line coverage
- Exclude generated code and trivial boilerplate
- Coverage on new / changed lines matters more than the total
- Mutation testing (sheet 25) checks what coverage can't

INTERVIEW ANGLE

- ? **Q:** "What does code coverage tell you?" — which code ran during tests, not whether it was verified
- ? Line vs branch: branch catches the untested `else`
- ? Good answer: use it to find gaps and as a floor on changed code, not as a target to game

LINE VS BRANCH COVERAGE

```
function fee(total, member):
    if total > 100 or member:          # condition with two parts
        return 0
    return 5

test "big orders are free":   assert fee(150, member=false) == 0

# line coverage:   2 of 3 lines    = 67%  ("return 5" never ran)
# branch coverage: 1 of 2 branches = 50% (the false branch never taken)
# condition coverage: "member" was never true → 50%

test "small orders pay 5":     assert fee(20, member=false) == 5
test "members ship free":      assert fee(20, member=true) == 0
# now 100% line, branch and condition coverage

# coverage can't see this bug:
test "does something":         fee(20, member=false)    # no assert, still "covered"
```

CODE EXPLAINED

- → Line coverage misses the untaken branch until a test hits `return 5`
- → Condition coverage forces a test where only `member` is true
- → The last test adds coverage while checking nothing
- → Report missing lines and branches, then decide which matter

IN YOUR STACK

- **pytest** pytest-cov `--cov-branch`, `--cov-fail-under` ↗18
- **Vitest** `--coverage` with v8 / istanbul, thresholds ↗20
- **JUnit** JaCoCo with coverage rules ↗23
- **Playwright** E2E coverage is rarely worth it; focus on unit / integration ↗40

SMELL CHECK

- **You see** a coverage target of 100% enforced → **it means** people write assertion-free tests to hit it → **do** set a sensible floor and review what's untested
- **You see** coverage rising while bugs keep shipping → **it means** coverage measures execution, not checking → **do** add mutation testing on critical code

COVERAGE KINDS

KIND	COUNTS	STRENGTH
Function	Functions called	Weak
Line / statement	Lines run	Basic
Branch	Each if / else side	Better
Condition	Each boolean sub-expression	Stronger
MC/DC	Each condition shown to matter	Safety-critical

USING COVERAGE WELL

DO	DON'T
Find untested code	Chase 100%
Floor on changed lines	Fail on tiny total dips
Branch coverage	Line coverage only
Review uncovered risky code	Write assert-free tests

⚠ COMMON MISTAKES

- Treating the coverage number as the goal
- Including generated code or vendored libraries in the measurement
- Ignoring which lines are uncovered and only reading the total

✓ TIPS & BEST PRACTICES

- Check the coverage diff in PRs: are the new lines tested?
- Turn on branch coverage everywhere
- Read uncovered lines in risky modules first

QUICK SUMMARY

- coverage — what ran, not what was checked
- branch > line
- floor on changed code, not a 100% target
- pair with mutation testing

Test Suite Health

KEEP THE SAFETY NET STRONG

A test suite is a product the team depends on, and it decays without care. Track a few health signals: **speed** (how long until feedback), **flake rate**, **pyramid shape** (are slow tests multiplying?), and failures that are **ignored** or **skipped**. Review tests in PRs as carefully as code, give suites **owners**, and delete tests that no longer earn their keep.

NOTES

- A suite people avoid running is already failing
- Most CI systems report slowest and flakiest tests
- Skipped tests rot; review them regularly
- Deleting a useless test is a valid improvement

A SUITE HEALTH REPORT

```
suite health — week 45
  total tests:          3,412   (+58)
  median CI time:       9m 40s  (target < 10m)
  slowest 10 tests:     4m 12s  combined   ← push down or parallelize
  flaky (retried):      7 tests (0.2%)     ← owners assigned
  skipped / disabled:   23                 ← 6 older than 90 days
  layer mix:            unit 81% · integration 16% · e2e 3%
  coverage on changed:  87%                (floor 80%)

PR review checklist for tests:
  [ ] does each new test fail if the feature breaks?
  [ ] names describe behavior
  [ ] no sleeps, no real network, no shared state
  [ ] the cheapest layer that catches the bug
```

CODE EXPLAINED

→ Trends matter more than single numbers

→ The slowest few tests often dominate suite time

→ Old skipped tests are either fixed or deleted

→ A short checklist keeps PR review of tests consistent

IN YOUR STACK

- **pytest** `--durations`, pytest-randomly, junit XML into CI dashboards ↗16
- **Vitest** slow-test reporting, projects split by speed ↗21
- **JUnit** Surefire / Gradle reports, Develocity test insights ↗23
- **Playwright** HTML / JSON reporters, flaky tracking ↗40

SMELL CHECK

- **You see** CI time creeping up every month → **it means** nobody owns speed → **do** set a time budget and fix the slowest tests
- **You see** dozens of skipped tests from last year → **it means** dead tests → **do** fix or delete them

HEALTH SIGNALS

SIGNAL	HEALTHY
PR feedback time	< ~10 min
Flake rate	< 0.5% of tests
Skipped tests	Few, all with issues
Layer mix	Mostly fast tests
Ignored failures	None

ACTIONS

PROBLEM	ACTION
Slow suite	Profile, parallelize, push tests down
Flakes	Quarantine with owners, fix causes
Ice-cream cone	Replace E2E edge cases with unit tests
Dead tests	Delete or rewrite
Unreadable tests	Factories, better names, smaller tests

⚠ COMMON MISTAKES

- Measuring only test count
- Letting "known failures" stay red in CI
- Nobody responsible for the suite as a whole

✓ TIPS & BEST PRACTICES

- Review test-health numbers in a regular team meeting
- Give the slowest-test list an owner
- Make deleting bad tests normal

QUICK SUMMARY

- speed · flake rate · layer mix · skips
- review tests like code
- owners for the suite
- delete tests that don't earn their keep

Tests in the Pipeline

pre-commit hooks · PR checks · stages (fast → slow) · fail fast · required checks · nightly suites

FAST FEEDBACK FIRST, THOROUGH CHECKS LATER

Arrange checks so the **fastest** run first and the **slowest** run least often. Pre-commit hooks catch formatting and lint in seconds. **PR checks** run unit and integration tests and must pass to merge (**required checks**). Slower suites (full E2E, performance, mutation) run on merge, nightly or before release. **Fail fast** so people get the first signal in minutes.

NOTES

- Target: PR feedback in under ~10 minutes
- Run tests affected by the change first where tools support it
- Make the CI command the same as the local one
- Nightly failures need an owner the next morning

A STAGED PIPELINE

```
# pre-commit (seconds, on the developer's machine)
- format + lint + type-check on changed files

# pull request (minutes, required to merge)
stages:
  - lint-and-types          # ~1 min, fail fast
  - unit-tests              # ~3 min, parallel
  - integration-tests       # ~5 min, real DB in a container
  - e2e-smoke               # ~3 min, critical journeys only

# main branch (after merge)
  - full e2e suite
  - build + deploy to staging
  - smoke tests against staging

# nightly
  - mutation tests on changed modules
  - performance / load tests
  - fuzzing (time-boxed)
```

CODE EXPLAINED

→ Cheap checks run first so obvious mistakes fail in a minute

→ Only the PR stages block merging

→ Smoke tests against the deployed environment catch config problems

→ Expensive suites still run, just less often

IN YOUR STACK

- **pytest** `pytest -x` locally, matrix in CI, `--junitxml` ↗23
- **Vitest** `vitest run --changed` in PRs, full on main ↗22
- **JUnit** `mvn verify` / `gradle check` with Failsafe ITs ↗23
- **Playwright** smoke tag on PR, full suite on main, sharded ↗63

SMELL CHECK

- **You see** a 45-minute PR pipeline → **it means** slow suites block every change → **do** move slow suites to post-merge or nightly
- **You see** nightly failures unread for a week → **it means** no owner → **do** alert a rotating owner

WHAT RUNS WHERE

STAGE	RUNS	BLOCKS MERGE?
Pre-commit	Format, lint, types	Local only
PR	Unit, integration, E2E smoke	Yes
Main	Full E2E, deploy, smoke	Blocks deploy
Nightly	Load, mutation, fuzz	No (alerts)

PIPELINE RULES

RULE	WHY
Fail fast	First signal in minutes
Same command locally and in CI	No "works on my machine"
Cache dependencies	Faster runs
Required checks	Main stays green
Upload artifacts on failure	Debug without rerunning (sheet 31)

⚠ COMMON MISTAKES

- Running everything on every commit until the pipeline is too slow to use
- Optional checks that people ignore
- CI commands that differ from what developers run locally

✓ TIPS & BEST PRACTICES

- Set a time budget for PR checks and defend it
- Run tests in parallel and shard large suites (sheet 30)
- Keep a small, reliable smoke suite for deploys

QUICK SUMMARY

- fast checks first, slow ones later
- PR checks required
- nightly for expensive suites
- same command locally and in CI

Parallelism & Sharding

MAKE BIG SUITES FAST

Parallelism runs tests at the same time on one machine (workers or threads); **sharding** splits the suite across several CI machines. Both need **isolated** tests: no shared files, ports, database rows or global state. Split by **timing** (balance by past duration) rather than file count for even shards, then **merge** the reports so failures show in one place.

NOTES

- Find order and isolation bugs before turning on parallelism
- Database per worker or unique data per test
- Too many workers can slow a resource-bound machine
- Timing data improves shard balance over time

FROM 40 MINUTES TO 8

```
before: 1 machine, 1 worker → 40 min

step 1 — parallel workers on one machine
    workers = cpu_count            # 4
    each worker: own DB schema "test_" + worker_id
    → 12 min

step 2 — shard across 4 CI machines
    shard i of 4 gets tests balanced by last run's timings
    each shard runs with 4 workers
    → ~3 min per shard (+ setup)

step 3 — merge results
    collect report-shard-1..4 → merge → one JUnit / HTML report
    → total wall time ≈ 8 min

isolation checklist:
    [ ] no fixed ports (use port 0 / random)
    [ ] temp files per test
    [ ] unique data or per-worker database
    [ ] no reliance on test order
```

CODE EXPLAINED

→ Workers need their own resources or they collide

→ Timing-based shards finish at about the same time

→ Merged reports keep failures in one view

→ Setup time per shard limits how far sharding helps

IN YOUR STACK

- **pytest** pytest-xdist `-n auto`, `worker_id`; pytest-split ↗19
- **Vitest** `pool`, `maxWorkers`, `--shard` + blob merge ↗21
- **JUnit** `junit.jupiter.execution.parallel.*`, build forking ↗16
- **Playwright** `workers`, `fullyParallel`, `--shard` + merge-reports ↗38

SMELL CHECK

- **You see** tests that pass serially but fail with workers → **it means** shared state → **do** give each worker its own resources
- **You see** one shard always finishing last → **it means** splitting by count → **do** split by timing

LEVELS OF PARALLELISM

LEVEL	ISOLATION	COST
Threads in one process	Weakest	Cheapest
Processes on one machine	Separate memory	Low
Shards on many machines	Separate machines	CI minutes + setup

SHARED-RESOURCE FIXES

RESOURCE	FIX
Database	Schema / DB per worker, or unique data
Ports	Bind to port 0
Files	Temp dir per test
Global config / env	Set and restore per test
External sandbox accounts	One account per worker

⚠ COMMON MISTAKES

- Turning on parallelism before fixing order-dependent tests
- Sharding by file count with very uneven file sizes
- Forgetting to merge shard reports

✓ TIPS & BEST PRACTICES

- Randomize order first; parallel second
- Store timing data from CI to balance shards
- Measure: more workers isn't always faster

QUICK SUMMARY

- parallel — workers on one machine
- sharding — split across machines
- isolation is the prerequisite
- split by timing, merge reports

Test Reports & Failure Triage

FROM RED BUILD TO ROOT CAUSE

When CI fails, you need the **what** (which test, which assertion), the **where** (environment, commit, shard) and the **why** (logs, traces, screenshots). **JUnit XML** is the common report format CI systems display. Save **artifacts** on failure: logs, Playwright traces, screenshots, coverage. Then **reproduce locally** with the same seed and command, and rerun only what failed.

NOTES

- Upload artifacts with "always" / "on failure" conditions
- Print the random seed and environment in the log header
- Annotations put failures on the PR's changed lines
- Triage first: is it the code, the test, or the environment?

TRIAGE STEPS

```
CI ✗ unit-tests (shard 2/4)

1. read the report, not the raw log
   FAILED tests/test_refunds.py::test_partial_refund
   assert 45.0 == 44.99
   seed: 8812   python 3.13   commit a1b2c3

2. classify
   code bug?        → recent change touched refunds.py   ← likely
   test bug?        → expectation still right?
   environment?     → only on CI? only on one shard?

3. reproduce locally with the same inputs
   run tests/test_refunds.py::test_partial_refund --seed 8812

4. E2E failure? open the trace artifact
   trace.zip → timeline, DOM snapshots, network, console

5. fix, push, rerun failed first
   rerun --last-failed → green → full run
```

CODE EXPLAINED

→ The test report names the test, the assertion and the values

→ Classifying first stops people from "fixing" a test that caught a real bug

→ The same seed and command reproduce the CI run

→ Traces show exactly what the browser saw at each step

IN YOUR STACK

- **pytest** `--junitxml`, `-ra`, `--lf` ↗23
- **Vitest** `junit` / `github-actions` reporters, `--last-failed` ↗22
- **JUnit** Surefire XML, Gradle HTML reports ↗23
- **Playwright** HTML report, traces on failure, trace viewer ↗43

SMELL CHECK

- **You see** people rerunning failed builds without looking → **it means** failures are noise → **do** require a triage note for reruns
- **You see** E2E failures with only "timeout" in the log → **it means** missing artifacts → **do** save traces and screenshots on failure

ARTIFACTS WORTH SAVING

ARTIFACT	SHOWS
Logs (app + test)	What happened
Traces (Playwright)	Step-by-step browser state
Screenshots / video	What the user saw
JUnit XML / HTML report	Which tests, which asserts
Seed + versions	How to reproduce

IS IT THE CODE, TEST OR ENVIRONMENT?

CLUE	POINTS TO
Fails locally with same seed	Code or test
Only fails on CI	Environment, resources, timing
Only one shard / worker	Shared state or order
Started after a dependency bump	Library change
Intermittent	Flaky (sheet 22)

⚠ COMMON MISTAKES

- Reading thousands of log lines instead of the test report
- Not saving artifacts, then being unable to reproduce
- Changing the test to match the new (wrong) behavior

✓ TIPS & BEST PRACTICES

- Put the seed, versions and commit at the top of every test log
- Turn on PR annotations for failing lines
- Write the triage outcome in the PR or issue

QUICK SUMMARY

- JUnit XML — the report CI reads
- artifacts — logs, traces, screenshots
- classify: code, test or environment
- reproduce with the same seed and command

Performance & Load Testing

load · stress · soak · spike · p50 / p95 / p99 · throughput · k6 · thresholds in CI

FAST ENOUGH UNDER REAL TRAFFIC

Functional tests ask "is it right?"; performance tests ask "is it fast enough, for how many users?" **Load** tests apply expected traffic, **stress** tests push past it to find the breaking point, **soak** tests hold load for hours to find leaks, and **spike** tests jump suddenly. Measure **latency percentiles** (p95, p99) and **throughput**, not averages, and set pass/fail **thresholds**.

NOTES

- Averages hide the slow requests users feel; use percentiles
- Test in an environment sized like production
- Ramp up gradually; record errors as well as latency
- Micro-benchmarks (one function) are a different tool from load tests

INTERVIEW ANGLE

- ? **Q:** "How would you load-test an API?" Define expected traffic, ramp up, measure p95 / p99 latency and error rate
- ? State a pass/fail threshold (e.g. p95 < 300 ms, errors < 1%)
- ? Mention a tool (k6, Gatling, Locust) and testing in a production-like environment

A LOAD TEST WITH THRESHOLDS

```
scenario "checkout API under expected peak":
    stages:
        ramp from 0 to 200 virtual users over 2 min
        hold 200 users for 10 min
        ramp down over 1 min
    each user loop:
        GET  /products?page=1
        POST /cart      { sku: random_sku() }
        POST /checkout  { cart_id }
        think 1-3 s
    thresholds:
        http_req_duration p95 < 300 ms
        http_req_duration p99 < 800 ms
        http_req_failed   rate < 1%

result:  p50 120 ms · p95 280 ms · p99 1.2 s ✗ · errors 0.4%
→ p99 fails: slow DB query on /checkout under load
```

CODE EXPLAINED

- → Ramping up shows where latency starts to climb
- → Think time makes virtual users behave like people
- → Thresholds turn the run into a pass / fail check for CI
- → p99 exposes the slow tail that the average hides

IN YOUR STACK

- **pytest** pytest-benchmark for micro-benchmarks; Locust for load ↗17
- **Vitest** `vitest bench` for micro-benchmarks; k6 for load ↗03
- **JUnit** JMH for micro-benchmarks; Gatling for load
- **Playwright** not a load tool; Lighthouse / web vitals for page performance

SMELL CHECK

- **You see** performance judged by the average response time → **it means** slow outliers are invisible → **do** report p95 / p99
- **You see** load tests run against a laptop-sized environment → **it means** results don't transfer → **do** test production-like capacity

TEST TYPES

TYPE	SHAPE	FINDS
Load	Expected traffic	Meets targets?
Stress	Beyond expected	Breaking point, failure mode
Soak	Long duration	Leaks, slow degradation
Spike	Sudden jump	Autoscaling, queues
Benchmark	One function	Regressions in hot code

METRICS

METRIC	MEANS
p50	Typical request
p95 / p99	Slow tail users feel
Throughput	Requests per second
Error rate	Failed requests %
Saturation	CPU, memory, connections

⚠ COMMON MISTAKES

- Using averages as the only latency number
- Load testing production without coordination
- Ignoring error rate while reading latency

✓ TIPS & BEST PRACTICES

- Start with a baseline run and compare every change against it
- Run a short load test in CI with thresholds, a long one nightly
- Profile the slowest endpoint before buying more servers

QUICK SUMMARY

- load · stress · soak · spike
- percentiles, not averages
- thresholds → pass / fail
- production-like environment

Accessibility Testing

CAN EVERYONE USE IT?

Accessibility testing checks that people using keyboards, screen readers, magnification or other assistive technology can use the product, against the **WCAG** guidelines (level AA is the usual target). **Automated** tools like axe catch missing labels, low contrast and invalid ARIA, roughly a third of issues. The rest needs **manual** checks: keyboard-only navigation, focus order, and screen reader spot checks.

NOTES

- Run automated checks in component and E2E tests
- Querying by role / label in tests is a free accessibility check
- Keyboard: every action reachable with Tab, Enter, Space, Esc, arrows
- The HTML, CSS and SwiftUI sets have Accessibility boxes on every sheet

AUTOMATED AND MANUAL CHECKS

```
# automated: run axe on each key page / component state
test "checkout page has no a11y violations":
    open_page("/checkout")
    results = axe.analyze(tags=["wcag2a", "wcag2aa"])
    assert results.violations == []

test "error state of login form is accessible":
    render(LoginForm, error="Wrong password")
    assert axe(container).violations == []
    expect(by_role("alert")).has_text("Wrong password")    # announced

# manual pass (per release, ~15 min per flow)
keyboard only:   Tab through checkout — order logical? focus visible?
                 open/close the dialog with Enter / Esc — focus returns?
screen reader:   VoiceOver / NVDA on the form — labels read? errors announced?
zoom 200%:       nothing cut off, no horizontal scrolling
```

CODE EXPLAINED

- → axe runs dozens of rules in milliseconds
- → Error states are tested too, not just the default page
- → Manual checks cover what tools can't judge: order, meaning, usability
- → Fix violations in components so every page using them benefits

IN YOUR STACK

- **pytest** axe via pytest-playwright for pages
- **Vitest** vitest-axe / jest-axe on rendered components ↗18
- **JUnit** axe-core Selenium / Playwright Java integration
- **Playwright** @axe-core/playwright, ARIA snapshots ↗36

SMELL CHECK

- **You see** a passing axe scan treated as "accessible" → **it means** two-thirds of issues unchecked → **do** add keyboard and screen reader passes
- **You see** tests that click by CSS selector on divs → **it means** non-semantic UI → **do** use buttons and roles; test by role

AUTOMATED VS MANUAL

AUTOMATION CATCHES	NEEDS A PERSON
Missing labels / alt text	Logical focus order
Low contrast	Meaningful labels and alt text
Invalid ARIA	Screen reader experience
Missing landmarks	Keyboard traps in complex widgets

WCAG LEVELS

LEVEL	MEANS
A	Minimum; serious barriers removed
AA	Usual legal and industry target
AAA	Highest; not required for whole sites

⚠ COMMON MISTAKES

- Testing only the happy path and ignoring error and loading states
- Adding ARIA to fix things native HTML would do
- Leaving accessibility to a pre-launch audit

✓ TIPS & BEST PRACTICES

- Add an axe check to every component test file
- Do a keyboard-only pass on each new flow
- Fix issues in shared components first

QUICK SUMMARY

- WCAG AA — the usual target
- axe — automated, ~1/3 of issues
- keyboard + screen reader — manual
- query by role — a11y check for free

Security Testing Basics

SAST · DAST · SCA / dependency scanning · secrets scanning · OWASP ZAP · security tests for auth rules

FIND VULNERABILITIES BEFORE ATTACKERS DO

Security testing combines automated scanners with your own tests. **SAST** analyzes source code for risky patterns; **SCA** finds known vulnerabilities in dependencies; **secrets scanning** catches committed keys; **DAST** attacks a running app from outside (OWASP ZAP). Just as important are **your own tests for authorization rules**: user A can't read user B's data, roles are enforced, input is validated.

NOTES

- Run SCA and secrets scanning on every PR
- Most real breaches are authorization bugs scanners can't understand
- Tie tests to the OWASP Top 10 categories
- Penetration tests by specialists still matter for high-risk apps

AUTHORIZATION TESTS + SCANNERS

```
# your tests: the rules only you know
test "a customer can't read another customer's order":
    res = client.get("/orders/" + bob_order.id, auth=token("ann"))
    assert res.status == 404                          # don't reveal it exists

test "price can't be set by the client":
    res = client.post("/orders", json={ sku: "A1", qty: 1, price: 0.01 }, auth=token("ann"))
    assert db.orders.find(res.body.id).price == catalog_price("A1")

test "search input is not executed as SQL":
    res = client.get("/search?q=' OR 1=1 --", auth=token("ann"))
    assert res.status == 200 and res.body.results == []

# pipeline scanners
pr:       secrets scan · dependency scan (SCA) · SAST
nightly:  DAST baseline scan against staging (OWASP ZAP)
```

CODE EXPLAINED

- → Broken object-level authorization is a top API risk; test it directly
- → Mass-assignment tests check that clients can't set server-owned fields
- → Injection tests send hostile input and check it is treated as data
- → Scanners cover known patterns and vulnerable libraries automatically

IN YOUR STACK

- **pytest** Bandit / Ruff `S` rules (SAST), pip-audit (SCA) ↗23
- **Vitest** `npm audit`, eslint-plugin-security, Snyk / Dependabot ↗22
- **JUnit** OWASP Dependency-Check, SpotBugs + Find Security Bugs ↗23
- **Playwright** auth tests per role, ZAP proxying Playwright traffic ↗31

SMELL CHECK

- **You see** only scanner results in the security review → **it means** authorization logic is untested → **do** write tests for who can access what
- **You see** dependency alerts piling up unread → **it means** alert fatigue → **do** auto-update and fail builds on critical issues

SECURITY TESTING KINDS

KIND	LOOKS AT	WHEN
SAST	Source code patterns	Every PR
SCA	Dependencies with known CVEs	Every PR + daily
Secrets scan	Committed keys / tokens	Pre-commit + PR
DAST	Running app from outside	Nightly / staging
Your auth tests	Access-control rules	Every PR
Pen test	Humans attacking	Before major releases

TESTS YOU SHOULD WRITE

RISK	TEST
Broken access control	User A → user B's resource = 404 / 403
Mass assignment	Client can't set role / price / owner
Injection	Hostile input stays data
Auth	Expired / tampered tokens rejected
Rate limits	Too many attempts → 429
Sensitive data	No secrets in responses or logs

⚠ COMMON MISTAKES

- Relying only on scanners for security
- Testing authorization with an admin account only
- Ignoring dependency alerts until an audit

✓ TIPS & BEST PRACTICES

- Write an access-control test for every new endpoint
- Fail CI on committed secrets
- Keep dependencies updated automatically with review

QUICK SUMMARY

- SAST · SCA · secrets · DAST — automated
- authorization tests — yours to write
- scan on every PR, DAST nightly
- OWASP Top 10 as a checklist

Testing Legacy Code

ADD A SAFETY NET BEFORE YOU CHANGE ANYTHING

Legacy code is code without tests (Michael Feathers' definition). Before changing it, pin its **current** behavior with **characterization tests**: run it, record what it does, assert exactly that, bugs included. Then find **seams** to break dependencies, add new code in tested pieces (**sprout**) or around old code (**wrap**), and refactor in small steps under the tests. A **golden master** captures large outputs wholesale.

NOTES

- Characterization tests describe what *is*, not what *should be*
- Fix bugs in separate, deliberate changes
- Start where you need to change code, not everywhere
- Book: *Working Effectively with Legacy Code*

INTERVIEW ANGLE

- ? **Q:** "How do you add tests to legacy code?" Characterization tests first, then seams, then refactor
- ? Mention sprout / wrap for new behavior
- ? Stress small, safe steps over a big rewrite

PINNING BEHAVIOR, THEN CHANGING IT

```
# 1. characterization: what does it do today?
test "legacy price calc — current behavior":
    assert legacy_price(qty=10, customer="gold") == 87.30     # observed, not designed
    assert legacy_price(qty=0,  customer="gold") == -2.70     # a bug! pinned for now

# 2. golden master for big outputs
for each input in recorded_inputs(1000):
    assert legacy_report(input) == read_golden(input.id)

# 3. sprout: new behavior in a new, tested function
function loyalty_discount(customer, total):   # TDD'd separately
    ...
legacy_price(...):
    total = old_logic(...)
    return total - loyalty_discount(customer, total)   # one-line change

# 4. refactor under the tests in small steps
#    extract method → run tests → rename → run tests → ...
```

CODE EXPLAINED

- → Characterization tests capture current behavior, including the negative price bug
- → A golden master over recorded inputs pins behavior that's too big to assert by hand
- → Sprouting keeps new logic tested even inside untested code
- → Each refactoring step runs the tests before the next one

IN YOUR STACK

- **pytest** golden files / syrupy over recorded inputs ↗22
- **Vitest** snapshot tests to pin current output ↗16
- **JUnit** ApprovalTests or `@TestFactory` over recorded cases ↗13
- **Playwright** visual snapshots to pin UI before a redesign ↗35

SMELL CHECK

- **You see** a big-bang rewrite of untested code → **it means** no way to know what broke → **do** characterize first, then change in small steps
- **You see** characterization tests "fixed" to expected behavior during refactoring → **it means** behavior changed silently → **do** change behavior only in separate, deliberate commits

TECHNIQUES

TECHNIQUE	USE WHEN
Characterization test	Before any change
Golden master	Output too big to assert
Sprout method / class	Adding new behavior
Wrap method / class	Adding behavior before / after old code
Extract and test	Pulling logic out step by step

COMMON SEAMS IN LEGACY CODE

SEAM	HOW
Parameterize constructor	Pass a dependency instead of new-ing it
Extract and override	Subclass in test, override the awkward call
Extract interface	Put an interface over a concrete dependency
Link / module seam	Swap a module at import / link time
Global → parameter	Pass what used to be global

⚠ COMMON MISTAKES

- Fixing bugs while adding characterization tests
- Refactoring without any tests in place
- Trying to test the whole legacy system before touching anything

✓ TIPS & BEST PRACTICES

- Add tests only around the code you're about to change
- Commit characterization tests separately from changes
- Use approval / golden tests to cover a lot of behavior quickly

QUICK SUMMARY

- characterization tests — pin what is
- seams — break dependencies
- sprout & wrap — new code, tested
- refactor in small steps under tests

Testing Time, Async & Concurrency

fake clocks · fake timers · awaiting vs polling · eventual consistency · race conditions · never `sleep`

MAKE TIME AND ORDERING TEST INPUTS

Code that depends on time, timers, background work or concurrency is the main source of slow and flaky tests. Control **time** with an injected clock or **fake timers**. **Await** async work directly; for work that completes later (queues, eventual consistency), **poll** for a condition with a timeout instead of sleeping. For races, make interleavings deterministic or run many iterations to expose them.

NOTES

- `sleep` in a test is a guess: too short = flaky, too long = slow
- Polling assertions retry until true or time out
- Fake timers advance time instantly
- Concurrency bugs need stress runs, thread-safety tools, or deterministic schedulers

TIME, ASYNC AND EVENTUAL CONSISTENCY

```
# time: inject a clock
test "session expires after 30 minutes":
    clock = FakeClock("2026-11-04T10:00:00Z")
    s = Session(clock); clock.advance(minutes=31)
    assert s.expired()

# timers: fake them
test "autosave fires once after typing stops":
    timers.fake(); editor.type("hello"); timers.advance(ms=999)
    assert saves == 0
    timers.advance(ms=1);  assert saves == 1

# async: await, don't sleep
test "fetches the profile":
    profile = await api.get_profile(7)
    assert profile.name == "Ann"

# eventual consistency: poll with a timeout
test "order event updates the search index":
    publish(OrderPlaced(id=7))
    eventually(timeout=5s): assert search("order 7").count == 1

# ✗ never
sleep(2); assert search("order 7").count == 1
```

CODE EXPLAINED

→ Advancing a fake clock tests a 30-minute rule in microseconds

→ Fake timers let a test check "not yet" and "now" exactly

→ Awaiting the promise / future means no guessing

→ `eventually` retries quickly and fails with a clear timeout

IN YOUR STACK

- **pytest** freezegun / time-machine, pytest-asyncio ↗20
- **Vitest** `vi.useFakeTimers`, `vi.setSystemTime`, `expect.poll` ↗14
- **JUnit** inject `Clock`, Awaitility for polling, `@Timeout` ↗15
- **Playwright** `page.clock`, auto-waiting web-first assertions ↗34

SMELL CHECK

- **You see** `sleep(5)` before an assertion → **it means** a timing guess → **do** await the work or poll for the condition
- **You see** tests that fail around midnight or month-end → **it means** real time leaks in → **do** inject a clock and test those dates on purpose

CONTROL TECHNIQUES

PROBLEM	TECHNIQUE
Business rules on dates	Inject a clock
Debounce / timers	Fake timers
Promises / futures	Await directly
Background jobs / events	Poll with timeout (eventually)
Races	Deterministic scheduling, stress loops

DATE EDGE CASES TO TEST

CASE	EXAMPLE
Month end	Jan 31 + 1 month
Leap day	2028-02-29
DST change	Local 02:30 that doesn't exist
Time zones	Same instant, different local date
Year end	Dec 31 23:59:59 → Jan 1

⚠ COMMON MISTAKES

- Sleeping instead of waiting for a condition
- Using the real clock in business-rule tests
- Testing concurrency once and declaring it safe

✓ TIPS & BEST PRACTICES

- Search the test suite for `sleep` and replace each one
- Test date edge cases explicitly with a fake clock
- Run concurrent code under stress loops in CI nightly

QUICK SUMMARY

- inject clocks, fake timers
- await async work directly
- poll with timeout for eventual results
- never `sleep` to wait

A Test Strategy for a Project

DECIDE WHERE TESTING EFFORT GOES

A **test strategy** answers: what are the **risks**, which **kinds of tests** cover them, at which **level**, and when they run. Start from risk: what breaks most often, costs the most, or would embarrass you? Put most effort there, at the cheapest level that catches the problem. Write the strategy down in a short page in the repo so the team shares the same picture.

NOTES

- Different projects get different shapes; a library ≠ a web app
- Revisit the strategy when incidents show gaps
- Include non-functional risks (performance, security, accessibility)
- One page is enough

INTERVIEW ANGLE

- ? **Q:** "How would you test this feature?" Start with risks, then map tests to levels using the pyramid
- ? Mention unit edge cases, an integration test at the real boundary, one E2E journey
- ? Add non-functional checks if relevant and say what you would *not* test

A ONE-PAGE STRATEGY (WEB SHOP)

```
# TESTING.md — Orders web app

Top risks
  1. wrong totals / tax / discounts          (money)
  2. checkout breaks after deploy            (revenue)
  3. one customer sees another's data        (security)
  4. slow checkout at peak                   (performance)

Coverage plan
  pricing rules        → unit tests + property tests       (risk 1)
  order API            → API tests incl. every error code  (risks 1, 3)
  DB / payments        → integration tests, containers     (risk 2)
  checkout journey     → 3 E2E tests, smoke after deploy   (risk 2)
  access control       → per-endpoint auth tests           (risk 3)
  peak traffic         → k6 load test nightly, p95 < 300ms (risk 4)

Not tested automatically
  marketing pages (manual review), admin CSV export layout

Runs
  PR: lint, unit, API, integration, E2E smoke   (< 10 min)
  nightly: full E2E, load, mutation on pricing
```

CODE EXPLAINED

- → Risks drive everything else in the document
- → Each test type is tied to the risk it covers
- → Saying what you won't test is part of the strategy
- → Run schedules match the cost of each suite

IN YOUR STACK

- **pytest** ↗01 — the strategy maps to pytest marks and CI jobs
- **Vitest** — projects per level (unit, dom, browser) ↗01
- **JUnit** — tags + Surefire / Failsafe split ↗01
- **Playwright** — smoke vs full suites with tags ↗01

SMELL CHECK

- **You see** testing effort spread evenly over all code → **it means** no risk analysis → **do** rank risks and focus there
- **You see** no written strategy → **it means** every developer guesses → **do** write one page and keep it current

SHAPES BY PROJECT TYPE

PROJECT	EMPHASIS
Web app	Unit + API + few E2E journeys
Public API	API + contract tests
Library	Unit + property tests across versions
Data pipeline	Data quality checks + golden datasets
Mobile app	Unit + UI tests on devices + manual

QUESTIONS TO ANSWER

QUESTION	EXAMPLE ANSWER
What could hurt most?	Money, data leaks, downtime
Cheapest test that catches it?	Unit vs integration vs E2E
When does it run?	PR, merge, nightly, release
Who owns it?	Team, rotation
What isn't tested?	And why that's acceptable

⚠ COMMON MISTAKES

- Copying another project's strategy without considering your risks
- A 30-page strategy nobody reads
- Never updating the strategy after incidents

✓ TIPS & BEST PRACTICES

- Start with a short risk list and grow from there
- Link each incident post-mortem to a strategy change
- Keep the strategy in the repo next to the code

QUICK SUMMARY

- risk first
- cheapest level that catches it
- say what you won't test
- one page, in the repo

Tests & AI Coding Agents

TESTS KEEP AI-WRITTEN CODE HONEST

AI coding agents write code quickly, and a good test suite is how you know it's right. **Tests act as the spec**: give the agent failing tests and let it make them pass. Review **generated tests** carefully: agents can write tests that mirror the implementation, assert nothing meaningful, or get "fixed" by weakening assertions. Keep **CI guardrails** (required checks, coverage on changed code, mutation testing) on for every change. The Agentic Coding set covers the tools.

NOTES

- Ask for tests first, review them, then ask for the implementation
- Watch for agents editing tests to make them pass
- Run the full suite on agent changes, not just new tests
- Playwright test agents can plan, generate and heal E2E tests

A TEST-FIRST PROMPT LOOP

```
# 1. you write (or approve) the failing tests — the spec
test "SAVE10 takes 10% off orders over $20"
test "SAVE10 is rejected under $20 with COUPON_MIN_TOTAL"
test "coupons can't be stacked"

# 2. prompt the agent
"Make the three failing tests in test_coupons.py pass.
 Don't modify the tests. Keep changes inside checkout/coupons.py.
 Run the full test suite before finishing."

# 3. review the diff — warning signs
- test files changed                    → reject unless asked
- assertions loosened (== → >=)         → reject
- new tests that copy the implementation's math → rewrite
- mocks of the code under test          → rewrite

# 4. guardrails in CI
required: full suite · coverage on changed lines ≥ 80%
nightly:  mutation testing on changed modules
```

CODE EXPLAINED

→ Human-reviewed tests define "done" before any code exists

→ Explicit constraints stop the agent from editing tests to pass

→ Diff review focuses on test changes first

→ CI checks apply to agent changes exactly as to human ones

IN YOUR STACK

- **pytest** ask for pytest tests first; review fixtures and asserts ↗22
- **Vitest** agent runs `vitest run` and reads failures ↗03
- **JUnit** OpenRewrite / agents for migrations, verified by tests ↗22
- **Playwright** test agents: planner, generator, healer ↗47

SMELL CHECK

- **You see** an AI change that also edited the failing tests → **it means** tests were bent to the code → **do** revert test edits and rerun
- **You see** generated tests with no failing-first step → **it means** they may pass for the wrong reason → **do** break the code and confirm they fail

REVIEWING GENERATED TESTS

CHECK	WHY
Did it fail first?	Proves it can fail
Assertions specific?	Catches weak checks
Mocks the code under test?	Tests nothing real
Copies implementation logic?	Repeats the bug
Names match behavior?	Readable spec

GUARDRAILS

GUARDRAIL	CATCHES
Required full suite	Breakage elsewhere
Protected test files / CODEOWNERS	Silent test edits
Coverage on changed lines	Untested new code
Mutation testing	Weak generated assertions
Small PRs	Reviewable diffs

⚠ COMMON MISTAKES

- Accepting agent-written tests without seeing them fail
- Letting an agent "fix" failures by editing assertions
- Running only the new tests on agent changes

✓ TIPS & BEST PRACTICES

- Write the test names yourself; let the agent fill in bodies, then review
- Tell the agent which files it may not change
- Use mutation testing on modules agents changed most

QUICK SUMMARY

- tests are the spec for agents
- review generated tests like code
- watch for loosened assertions
- CI guardrails on every change

Sheet 39

Quick Reference — Test Types & Doubles

test types table · pyramid layers · five doubles · when to mock · tool per layer in each stack

TEST TYPES (02, 03)

TYPE	SCOPE / PURPOSE	SPEED
Unit	One function / class	ms
Integration	Parts + real infra	10 ms–1 s
End-to-end	Whole app via UI / API	Seconds
Contract	API agreement between services	Seconds
Smoke	Is it alive after deploy?	Seconds
Regression	Old bug stays fixed	Any
Acceptance	Requirement met	Any
Performance	Fast enough under load	Minutes

TEST DOUBLES (10, 11)

DOUBLE	DOES	VERIFY
Dummy	Fills a parameter	—
Stub	Canned answers	State
Fake	Simplified working version	State
Spy	Records calls	Interaction
Mock	Expectations up front	Interaction

MOCK OR REAL? (11, 12)

DEPENDENCY	IN UNIT TESTS
Your domain objects	Real
Value objects / DTOs	Real
Database / repository	Fake (real in integration)
HTTP to other services	Fake at HTTP level
Clock / random / IDs	Injected fakes
Email / payments	Mock or fake your port
Third-party SDK	Wrap in a port

TOOL PER LAYER

LAYER	PYTEST	VITEST	JUNIT	PLAYWRIGHT
Unit	pytest	Vitest (node)	JUnit + AssertJ	—
Component	Django client	Testing Library	@WebMvcTest	component tests
Integration	fixtures + containers	globalSetup + containers	@SpringBootTest + Testcontainers	API tests
E2E	pytest-playwright	browser mode (components)	Playwright for Java	Playwright Test
Mocking	monkeypatch / pytest-mock	vi.fn / vi.mock / MSW	Mockito	page.route

GOOD TEST CHECKLIST (04, 05)

CHECK	MEANS
Fast	ms per unit test
Isolated	No order dependence
Repeatable	No real clock / network / randomness
Self-validating	Pass / fail by itself
Behavior-focused	Survives refactors
One Act	One reason to fail
Clear name	Reads as a requirement

PYRAMID SHAPES (02)

SHAPE	EMPHASIS
Pyramid	Mostly unit
Trophy	Static + integration (front end)
Honeycomb	Integration (microservices)
Ice-cream cone	Anti-pattern: mostly E2E / manual

Sheet 40

Quick Reference — Techniques & Smells

technique picker (property · snapshot · mutation · fuzz · contract) · smell → fix table · CI checklist

TECHNIQUE PICKER (19, 23–26)

NEED	TECHNIQUE
A rule for all inputs	Property-based
Large output to pin	Snapshot / approval
Are my tests strong?	Mutation testing
Untrusted input parsing	Fuzzing
Services agree on an API	Contract testing
Pin legacy behavior	Characterization / golden master
Requirements with stakeholders	BDD / Gherkin

SMELL → FIX (09, 22)

SMELL	FIX
Fragile	Test behavior, not implementation
Flaky	Remove time / order / network dependence
Slow	Push down the pyramid
Mystery guest	Build data in the test
Eager test	One behavior per test
Logic in test	Parameterize
Over-specified	Assert outcomes
Can't fail	Break the code; watch it go red
sleep()	Await or poll with timeout

CI CHECKLIST (29–31)

ITEM	RULE
PR feedback	< ~10 minutes
Order	Lint → unit → integration → E2E smoke
Required checks	Main stays green
Parallel / shards	Isolated tests, split by timing
Artifacts	Logs, traces, screenshots on failure
Reports	JUnit XML, merged
Nightly	Full E2E, load, mutation, fuzz

COVERAGE & STRENGTH (25, 27)

MEASURE	TELLS YOU
Line coverage	Which lines ran
Branch coverage	Which if / else sides ran
Coverage on changed lines	New code tested?
Mutation score	Do tests notice changes?
Flake rate	Can you trust red?
Suite time	Will people run it?

NON-FUNCTIONAL (32–34)

AREA	CHECK
Performance	p95 / p99 + error rate thresholds
Accessibility	axe + keyboard + screen reader
Security	SCA, secrets, SAST, DAST + auth tests

TDD & BDD (13–15)

PRACTICE	CORE LOOP
TDD	Red → green → refactor
Outside-in	Failing acceptance test → inward
BDD	Examples → Given / When / Then → steps

AI AGENT GUARDRAILS (38)

RULE	WHY
Tests first, reviewed	Spec before code
Don't edit tests	No bending tests to code
Full suite required	Catch side effects
Mutation on changed code	Catch weak asserts